
Republic of Tunisia

Ministry of Higher Education and Scientific Research

University of Tunis El Manar

Faculty of Sciences of Tunis


End of Studies Project

***National Diploma of Engineering in computer engineering***

---

# STM32Cube Competitors benchmark

---

Presented by : Fares DKHILI

date :


President :

Reporter :

Academic supervisor :

Professional supervisor :


End of studies project presentation

by

**Fares DKHILI**

# STM32Cube Competitors benchmark

Professional Supervisor : **Mme. Sirine El Jazi**

# PLAN


I General Context

II Problem Statement and Objectives

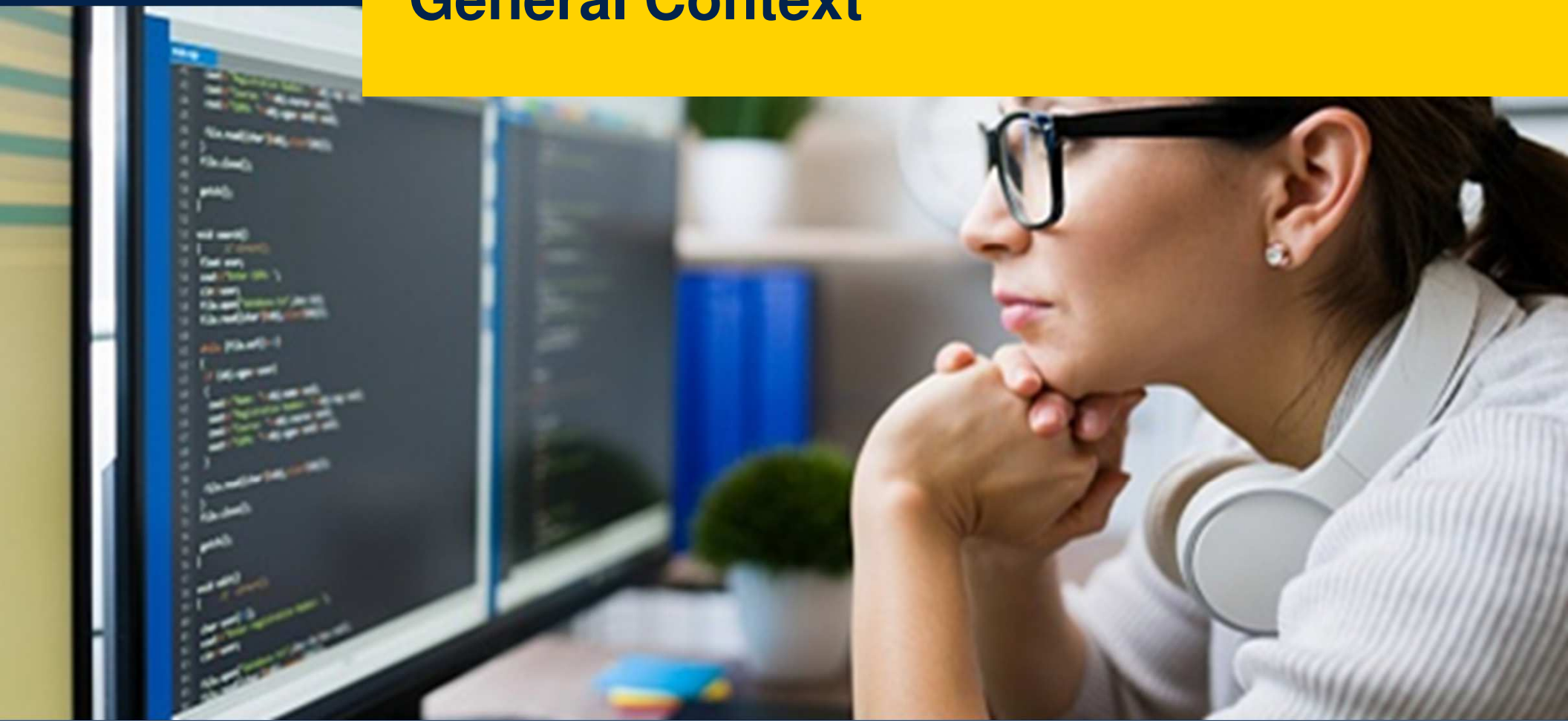
III Benchmark Methodology

IV Work Carried Out

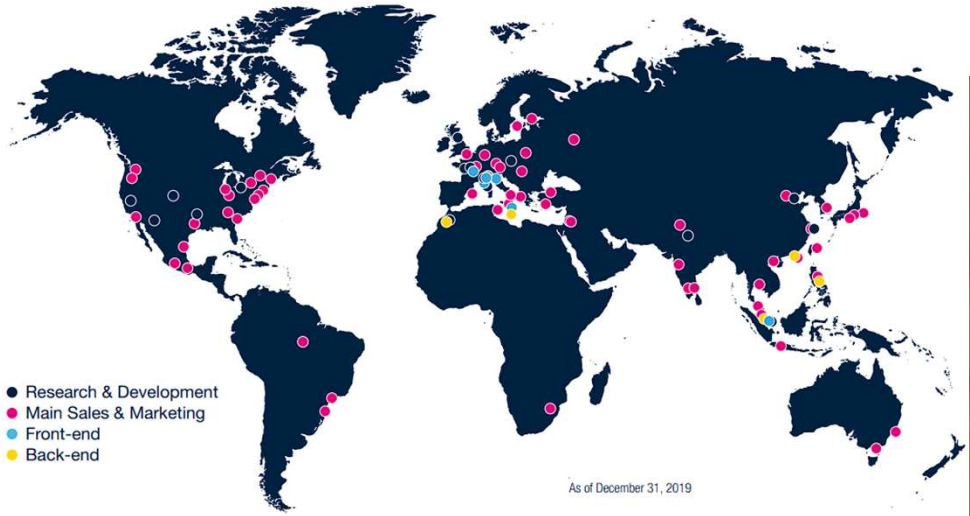
V Results and Contributions

VI Conclusion and Perspectives

# General Context



## STMicroelectronics




One of the world's largest semiconductor companies

**50,000** employees
of which **9,000+** in R&D

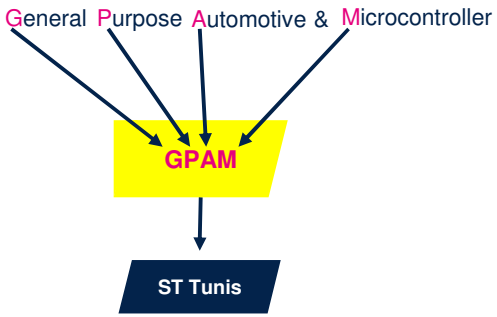
**$13.3 billion**
revenues in 2024

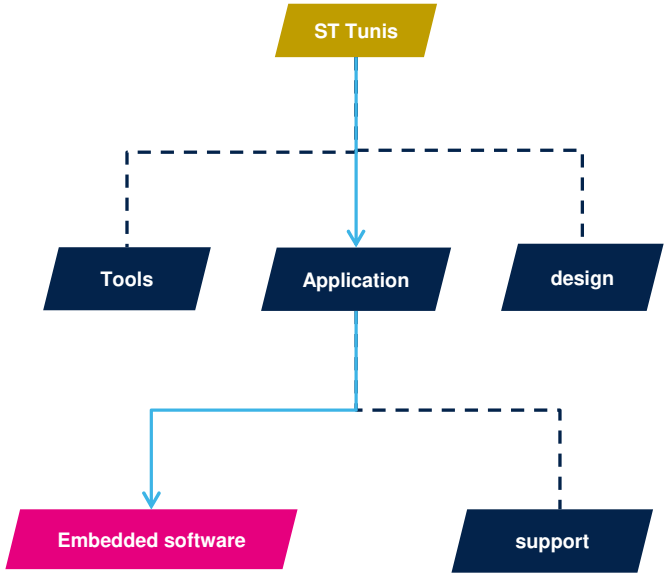
Over **80** sales & marketing offices serving over **200,000** customers across the globe

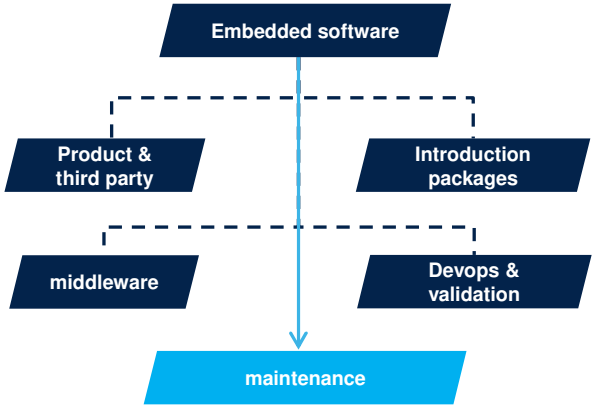
**14** main manufacturing sites

Signatory of the United Nations Global Compact (UNGC)
Member of the Responsible Business Alliance (RBA)

## ST Tunis


General Purpose Automotive & Microcontroller
GPAM
ST Tunis


ST Tunis
Tools
Application
design
Embedded software
support


Embedded software
Product & third party
Introduction packages
middleware
Devops & validation
maintenance

## Why competitor benchmark matters?

Benchmarking STM32Cube against competitor vendors to demonstrate strengths and guide improvements

### CORE PURPOSE

**Compare ST with competitor vendors, highlight where STM32Cube is stronger, identify gaps, and translate findings into concrete enhancement proposals.**

**⇄ Compare**

Evaluate ST and competitors under the same benchmark scenarios and fair conditions.

**★ Highlight Strengths**

Show where STM32Cube performs better in footprint, execution, maturity, or software quality.

**⚠ Identify Gaps**

Detect where competitors are stronger and understand the technical origin of the difference.

**↑ Improve**

Turn benchmark findings into targeted proposals at HAL, middleware, configuration, or integration level.

### KEY TAKEAWAY

**This project helps ST prove where STM32Cube is strong and know exactly what should be improved where competitors perform better.**

# Problem Statement and Objectives

# Problem statement

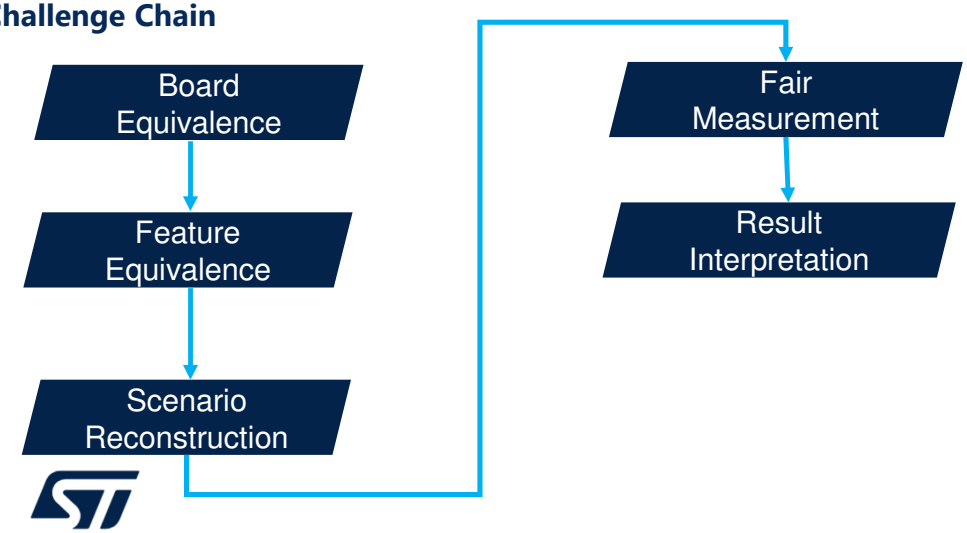
Why competitor benchmarking is more complex than a simple comparison

## Main Problem

**Benchmarking competitor solutions is not straightforward because a fair comparison requires much more than running two projects side by side.**

- **Selecting relevant equivalent boards**
- **Identifying matching software components**
- **Rebuilding representative benchmark scenarios**
- **Generating reliable and exploitable measurements**

## Challenge Chain




## Additional Complexity

- Some legacy benchmark scenarios already existed
- These scenarios had to be **rebuilt, updated, and improved**
- The benchmark scope also had to be **extended to new competitors**

# Objectives

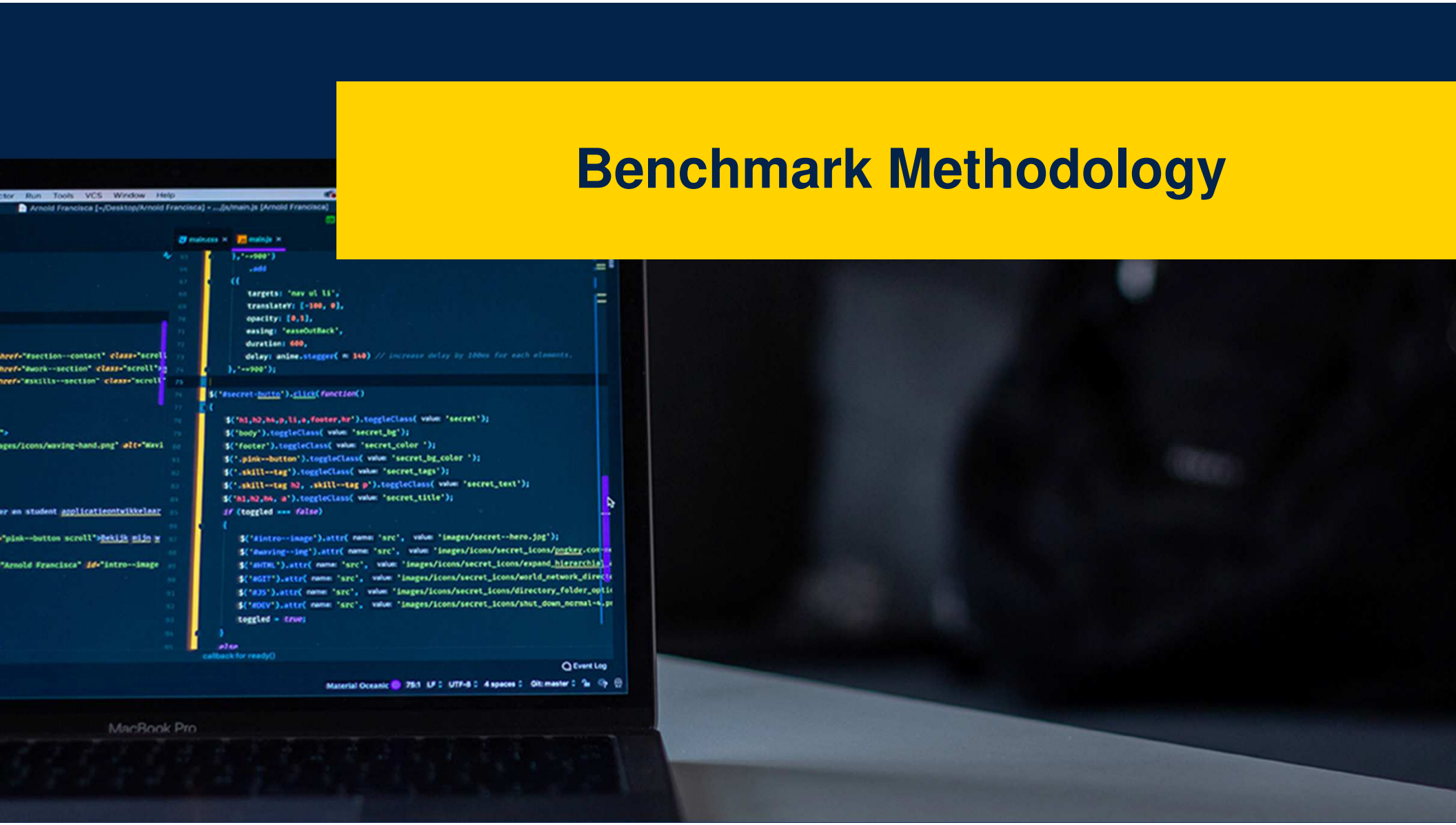
## Main Benchmark Objectives

- Identify relevant competitor boards and software equivalences
- Recreate and improve existing benchmark scenarios
- Regenerate updated footprint and execution-time analyses
- Extend benchmark activity toward new comparison cases

## Contribution Objectives

- Contribute effectively to benchmark execution
- Improve benchmark workflow efficiency through support developments
- Facilitate result analysis and scenario preparation

# Benchmark Methodology



## Workflow Process Stages

**Plan** 01

Teams outline short-term goals and deliverables for the defined duration.

**Build** 02

Create benchmark Scenarios and build them for both vendors.

**Extract** 03

Extract KPIs from projects (footprint, performance etc..)

**Interpret** 04

Data analysis and interpretation and enhancement proposals.

**Deliver** 05

Final output is released to marketing team for immediate feedback and use.

## Slide 12

**FD1** Icons

Fares DKHILI, 2026-07-06T15:22:46.337

## Evaluation criteria and benchmark dimensions

The benchmark is multi-dimensional and not limited to a single metric

### Static Analysis Orientation

*Code structure and ecosystem quality view*

- composition
- Dependency behavior
- API surface
- documentation
- duplication
- complexity
- compliance
- coverage

### Functional Analysis orientation

*Feature-level comparison and implementation behavior*

- Focuses on scenario execution under equivalent observable behavior
- Includes both footprint and execution-time perspectives

**Footprint Analysis**

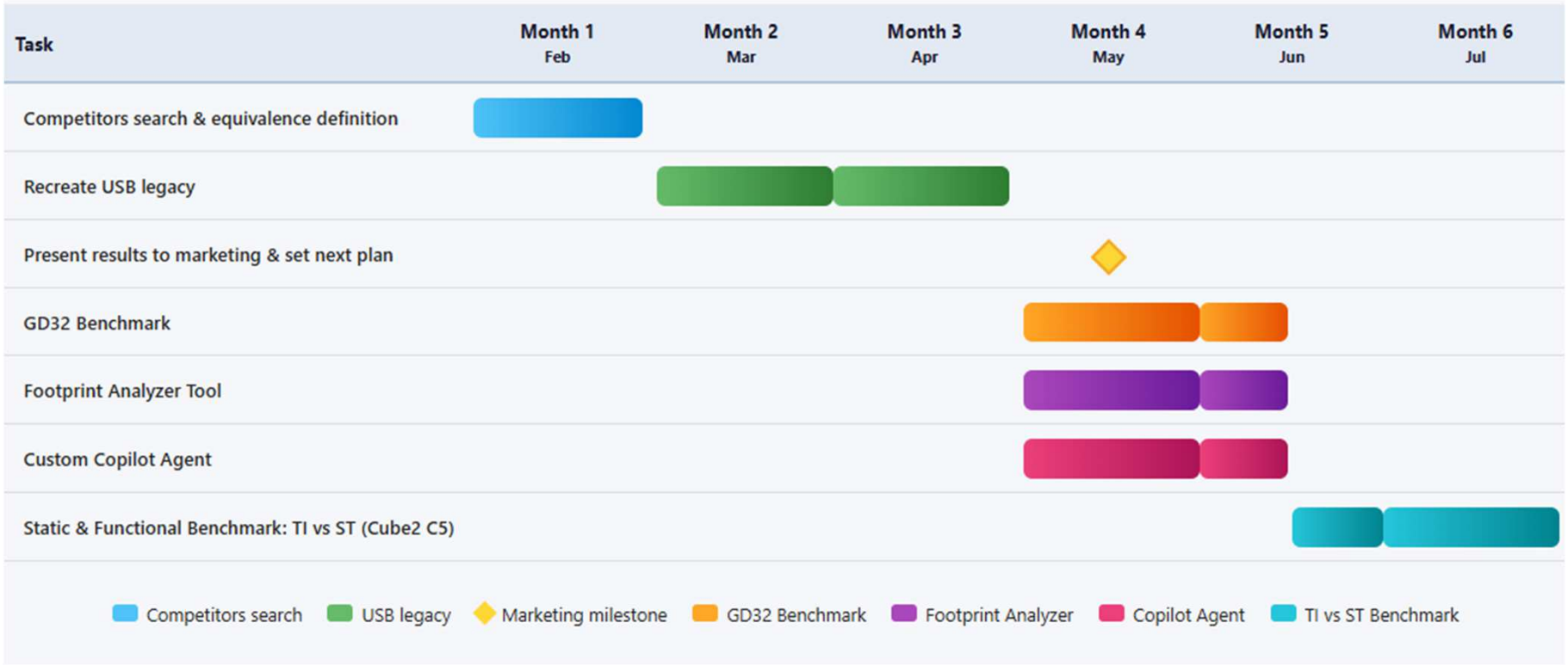
- Code size
- Memory occupation
- Per-layer
- per-module

→ Interpret gaps and propose optimization

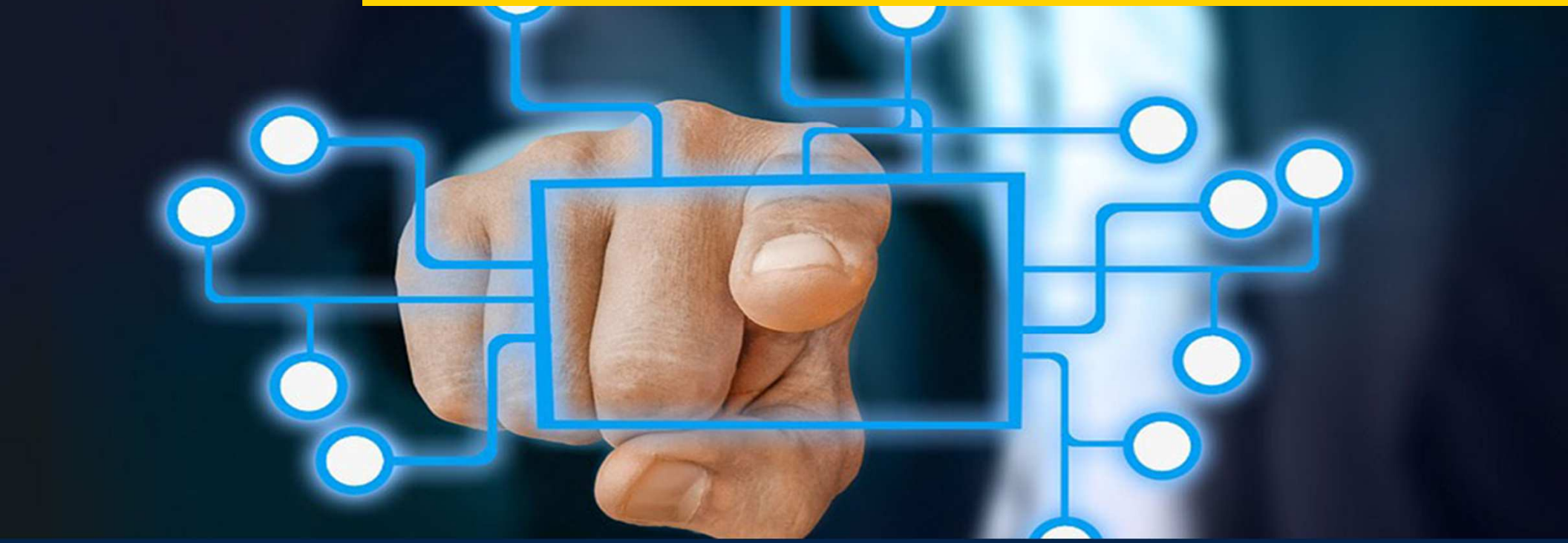
**Performance Analysis**

- Bring up
- Enumeration
- Data Transmission
- Threads/s

## Gantt Diagram



# Work Carried Out



# Competitor Search and Equivalence Definition

## Preparation Activities

- Identify the closest **ST equivalent targets**
- Study possible **benchmark scenarios**
- Establish the **comparison basis** for future measurements

## Goal of This Phase

- Search for available **competitor boards** and ecosystems
- Identify the closest **ST equivalent targets**
- Study possible **benchmark scenarios**
- Establish the **comparison basis** for future measurements

**01 Explore Competitors**

Identify vendor families, boards, MCU classes, and available software ecosystems.

**02 Match ST Targets**

Define hardware and software equivalence to ensure comparable benchmark conditions.

**04 Select Scenarios**

Evaluate which software stacks and benchmark cases are relevant and feasible.

**05 Build Comparison Basis**

Prepare a structured basis for fair execution, interpretation, and future reuse.

# Recreation of Legacy USB Benchmark Scenarios

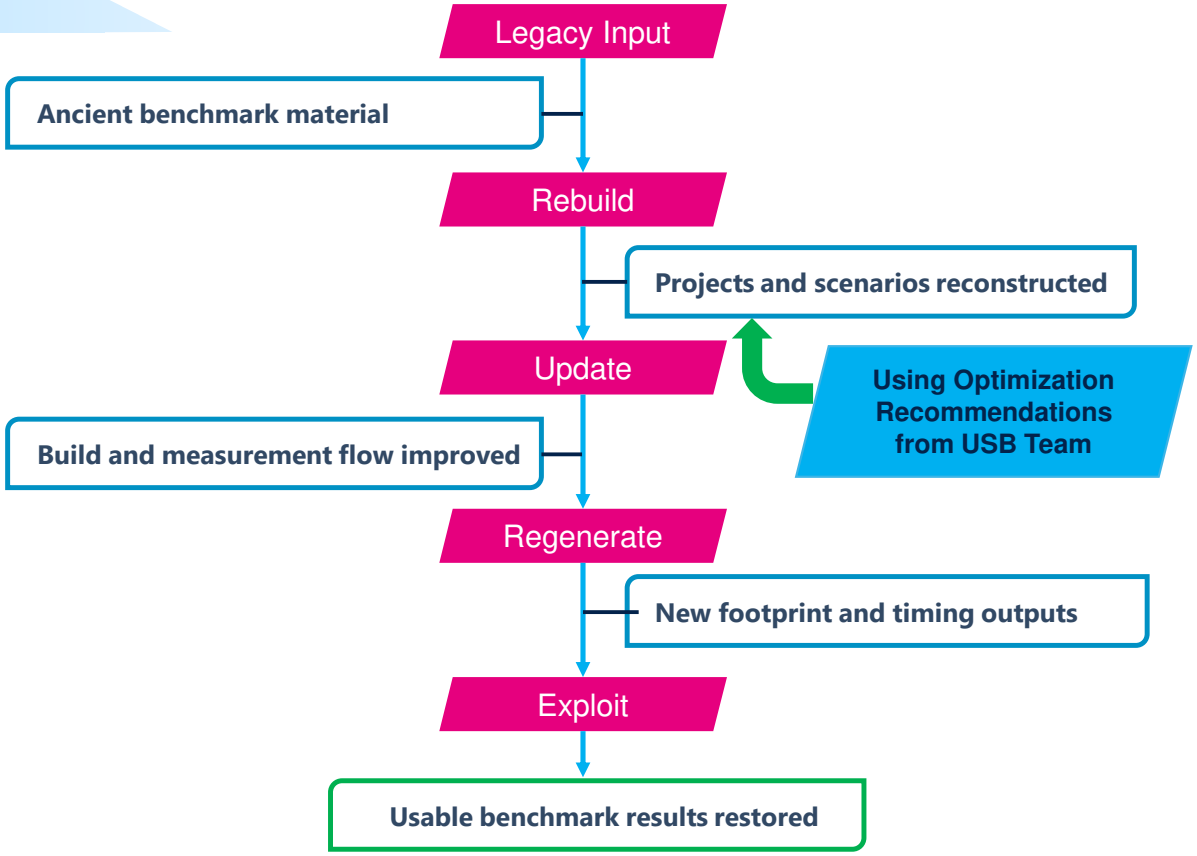
Reviving and modernizing existing benchmark assets

**Recreated Scenarios**

- **Renesas USB** vs **ST USB stack**
- **NXP USB** vs **ST USB stack**

**Core purpose:**

**Cleaning ancient projects and deliver Deterministic Results after optimization for better review.**



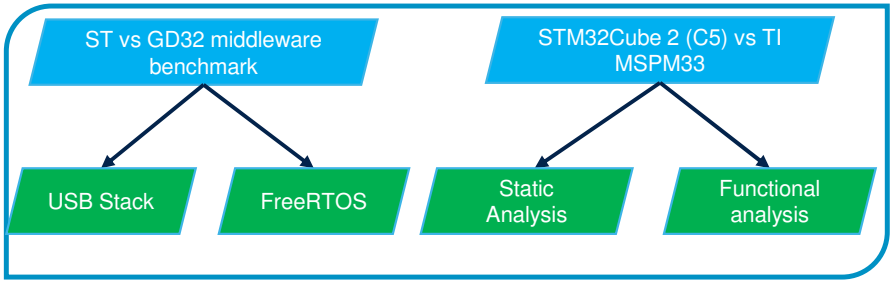
# Internal Feedback and Benchmark Roadmap Evolution

## Initial Outputs

- Legacy USB benchmark results consolidated internally
- Benchmark material prepared for internal presentation
- First comparison results reused in a broader effort

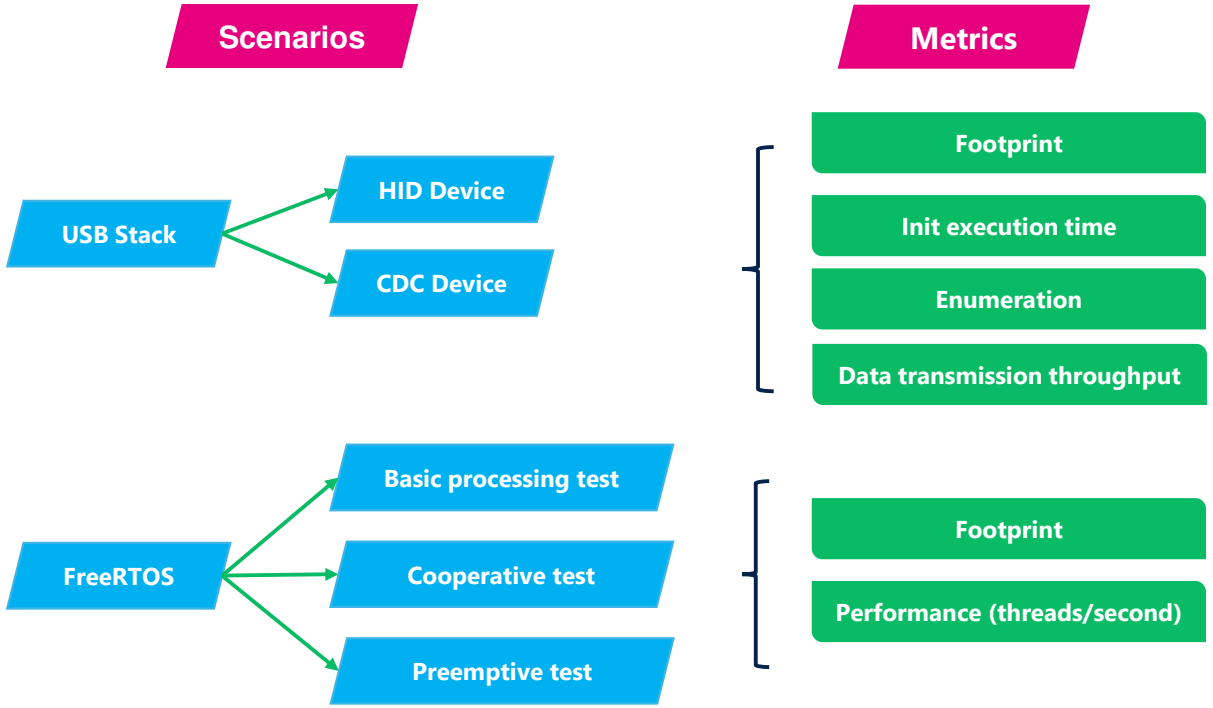
**Stakeholder Feedback:**
**Benchmark results were not isolated outputs. They were presented, discussed, and used to refine priorities and future comparison directions.**

## New Benchmark Plan



## GD32 Benchmark Scope

*Completed benchmark scenarios, measured metrics, and supporting tools used in the campaign*

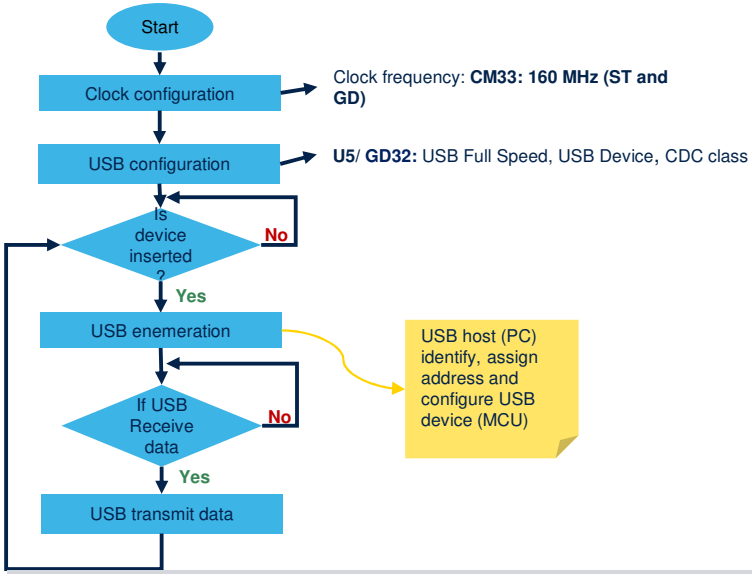


# GD32 Benchmark Scenarios Specification

USB Middleware

## BareMetal mode : ST/GD USB Library CDC device class

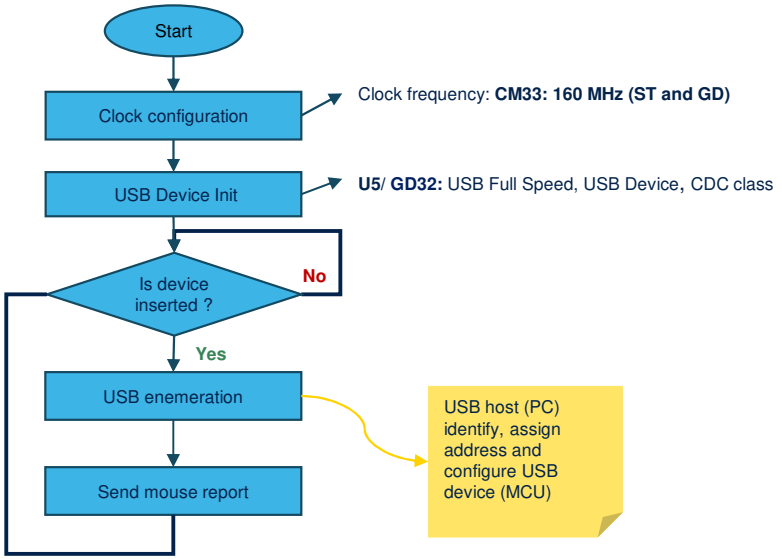
**STM32U575xx (CM33) vs GD32E507(CM33)**




- **USB CDC device class:** USB **communication device class** allows devices to communicate with computers as serial ports or network interfaces.

## BareMetal mode : ST/GD USB Library USB HID device class

**STM32U575xx (CM33) vs GD32E507(CM33)**




- **USB HID device class:** USB **Human Interface Device** allows to enable communication between **device (MCU)** and host (PC).

# STM32 vs GD32 MW Benchmark: USB stack

**Test Results**

## STM32/GD32 USB Library CDC device (FS)

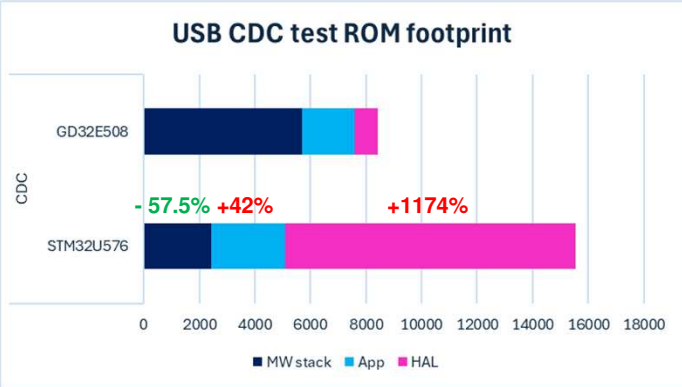
**Rule: STM32 vs GD32 (%) = (STM32– GD32) / GD32**

### Footprint (Bytes)

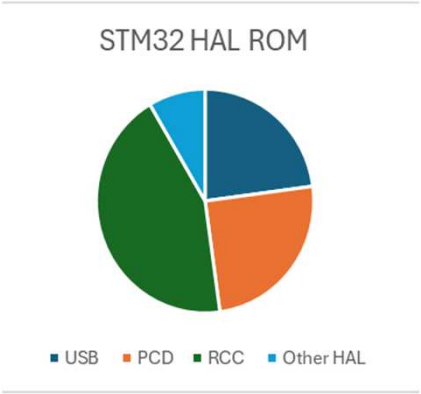
| CDC Test Footprint | STM32U575 | GD32E507 | STM232 vs GD32 |
|---|---|---|---|
| **ROM** | 17,049 | 9,085 | **+ 88 %** |
| **RAM** | 4,089 | 9,683 | **- 57.7 %** |

### ROM Footprint (Bytes)

| ROM | STM32U575 | GD32E507 | STM232 vs GD32 |
|---|---|---|---|
| **App** | 2,683 | 1,882 | **+ 42.5%** |
| **HAL** | 10,446 | 820 | **+ 1174%** |
| **MW stack** | 2,423 | 5,709 | **- 57.5 %** |




- STM32 ROM footprint is higher by average 88% than GD32
- This is due to:
  - **Application ROM:** STM32 higher by 42% than GD32 because of USB Device user files: usb_device.c, usbd_cdc_if.c, usbd_desc.c, usbd_conf.c (1164 bytes: 43% of total App ROM footprint). In GD32, USB device configuration is managed directly in MW stack
  - ➢ More USB Device configuration flexibility in STM32
  - **HAL ROM:** STM32 higher by 1174% than GD32, mainly due to:
    - HAL RCC: 4560 bytes => 44% of total HAL ROM footprint
    - HAL/LL USB: 5006 bytes => 48% of total HAL ROM footprint
- The high STM32 App and HAL ROM footprint lead to a lower STM32 USB library ROM footprint than GD32 USB library by 58%




### Usb cdc device operations (ms)

| USB CDC Device operations(ms) | STM32U575 | GD32E507 | STM232 vs GD32 |
|---|---|---|---|
| **Initialization** | 9.9936 | 26.07624 | **-61.68%** |
| **Enumeration** | 218.2 | 218.5 | **-0.14%** |
| **Transmit data** | 0.00988 | 0.010034 | **-1.53%** |
| **Receive data** | 0.00986 | 0.009932 | **-0.72%** |

- The execution time of STM32U5 USB initialization phase is lower by 61.68% than GD32E5
- The execution time of enumeration, transmission and reception phases are almost close between STM32U5 and GD32E5.

# STM32 vs GD32 MW Benchmark: USB stack

**Test Results**

## STM32/GD32 USB Library HID device (FS)

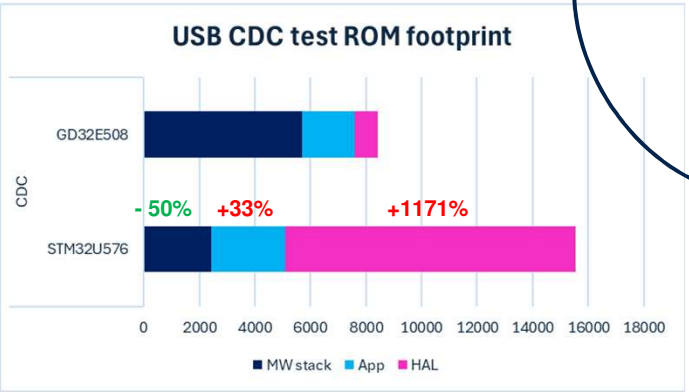
**Rule: STM32 vs GD32 (%) = (STM32– GD32) / GD32**

### Footprint (Bytes)

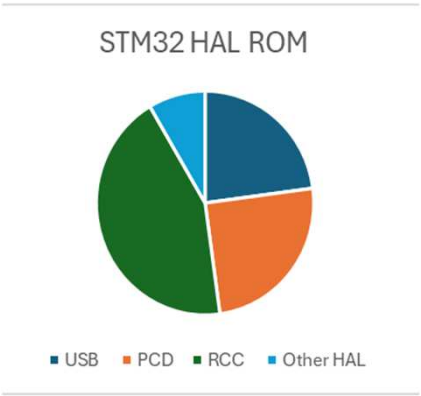
| HID Test Footprint | STM32U575 | GD32E507 | STM232 vs GD32 |
|---|---|---|---|
| **ROM** | 16,269 | 9,175 | + 77 % |
| **RAM** | 3,446 | 9,608 | - 65 % |

### ROM Footprint (Bytes)

| ROM | STM32U575 | GD32E507 | STM232 vs GD32 |
|---|---|---|---|
| **App** | 2580 | 1942 | + 33 % |
| **HAL** | 10426 | 820 | + 1171 % |
| **MW stack** | 2859 | 5743 | - 50 % |




- STM32 ROM footprint is higher by average 77% than GD32
- This is due to:
  - **Application ROM:** STM32 higher by 33% than GD32 because of USB Device user files: usbd_desc.c, usbd_conf.c, (985 bytes: 38% of total App ROM footprint). In GD32, USB device configuration is managed directly in MW stack
    - ➢ More USB Device configuration flexibility in STM32
  - **HAL ROM:** STM32 higher by 1171% than GD32, mainly due to:
    - HAL RCC: 4560 bytes => 44% of total HAL ROM footprint
    - HAL/LL USB: 4988 bytes => 48% of total HAL ROM footprint
- The high STM32 App and HAL ROM footprint lead to a lower STM32 USB library ROM footprint than GD32 USB library by 50%




**RAM consumption**

- RAM footprint analysis shows that the GD32 implementation requires significantly more stack than STM32.
- For both HID and CDC examples, **GD32 needs 0x2000 bytes of stack and 0x2000 bytes of heap**, whereas **STM32 requires only 0x400 bytes of stack and 0x200 bytes of heap** for HID, and the same order of magnitude for CDC.

### Usb cdc device operations (ms)

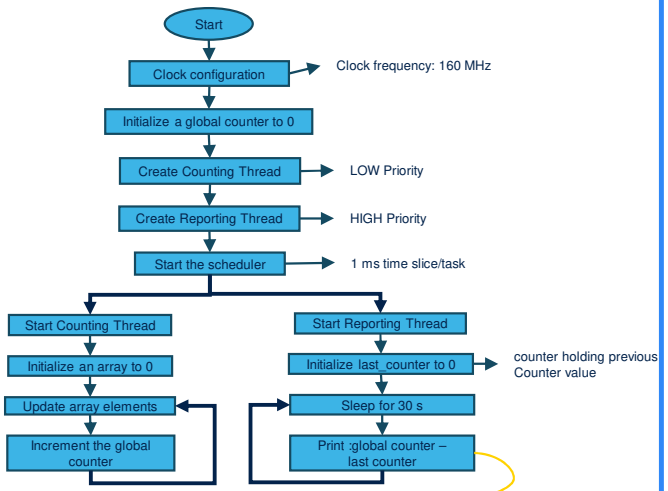
| USB HID Device operations | STM32U575 | GD32E507 | STM232 vs GD32 |
|---|---|---|---|
| **Initialization** | 9.990106 | 26.08368 | -61.70% |
| **Enumeration** | 213 | 216 | -1.39% |
| **Data transmission** | 0.009284 | 0.00935 | -0.71% |

- The execution time of STM32U5 USB initialization phase is lower by 61.70 % than GD32E5
- The execution time of enumeration and data transmission phases are almost close between STM32U5 and GD32E5.

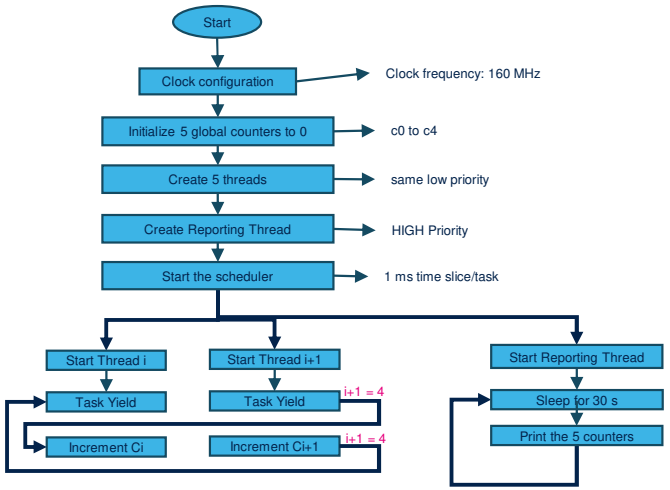
# GD32 Benchmark Scenarios Specification

FreeRTOS

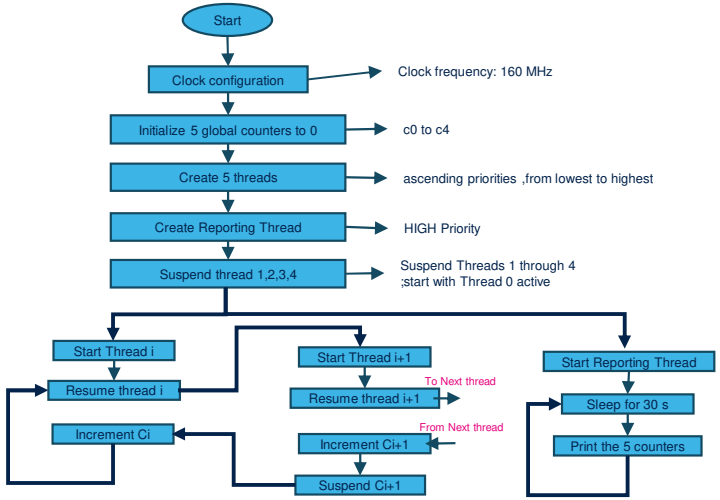
## Basic Processing Flowchart




## Cooperative Processing Flowchart




## Preemptive Scheduling Flowchart



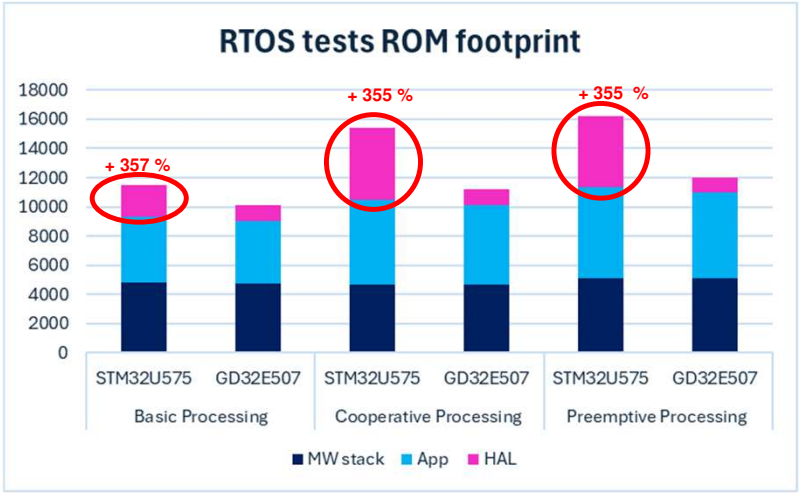
# STM32 vs GD32 MW Benchmark: FreeRTOS

**FreeRTOS tests Footprint(bytes)**

| RTOS Scenario | Footprint | STM32U575 | GD32E507 | STM232 vs GD32 |
|---|---|---|---|---|
| Basic Processing | ROM | 14266 | 10128 | + 40.86 % |
| | RAM | 11960 | 11716 | +2.08 % |
| Cooperative Processing | ROM | 15422 | 11224 | + 37 % |
| | RAM | 10972 | 10728 | +2.27 % |
| Preemptive Processing | ROM | 16226 | 12028 | + 35 % |
| | RAM | 10996 | 10748 | + 2.31 % |

**FreeRTOS tests ROM Footprint (bytes)**

| RTOS Scenario | Footprint | STM32U575 | GD32E507 | STM232 vs GD32 |
|---|---|---|---|---|
| Basic Processing ROM | App | 4531 | 4278 | + 6 % |
| | HAL | 4927 | 1078 | + 357 % |
| | MW stack | 4808 | 4772 | + 0.7 % |
| Cooperative Processing ROM | App | 5823 | 5484 | + 6 % |
| | HAL | 4911 | 1078 | + 355 % |
| | MW stack | 4688 | 4662 | + 0.55 % |
| Preemptive Processing ROM | App | 6191 | 5810 | + 6 % |
| | HAL | 4911 | 1078 | + 355 % |
| | MW stack | 5124 | 5140 | -0.3 % |

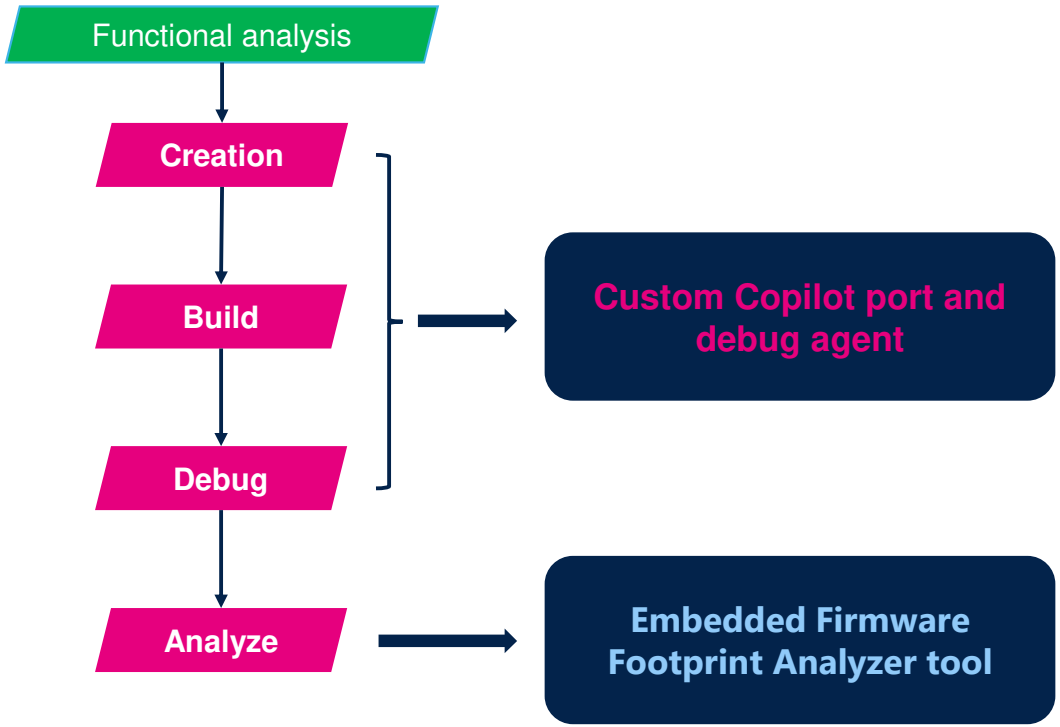



**FreeRTOS tests performance (thread/s)**

| RTOS Scenario | STM32U575 | GD32E507 | STM232 vs GD32 |
|---|---|---|---|
| Basic Processing (Thread/s) | 47,943 | 47,955 | - 0.025 % |
| Cooperative Processing (Thread/30s) | 1,220,066 | 1,200,666 | + 1.6 % |
| Preemptive Processing (Thread/30s) | 294,615 | 285,144 | + 3.3 % |

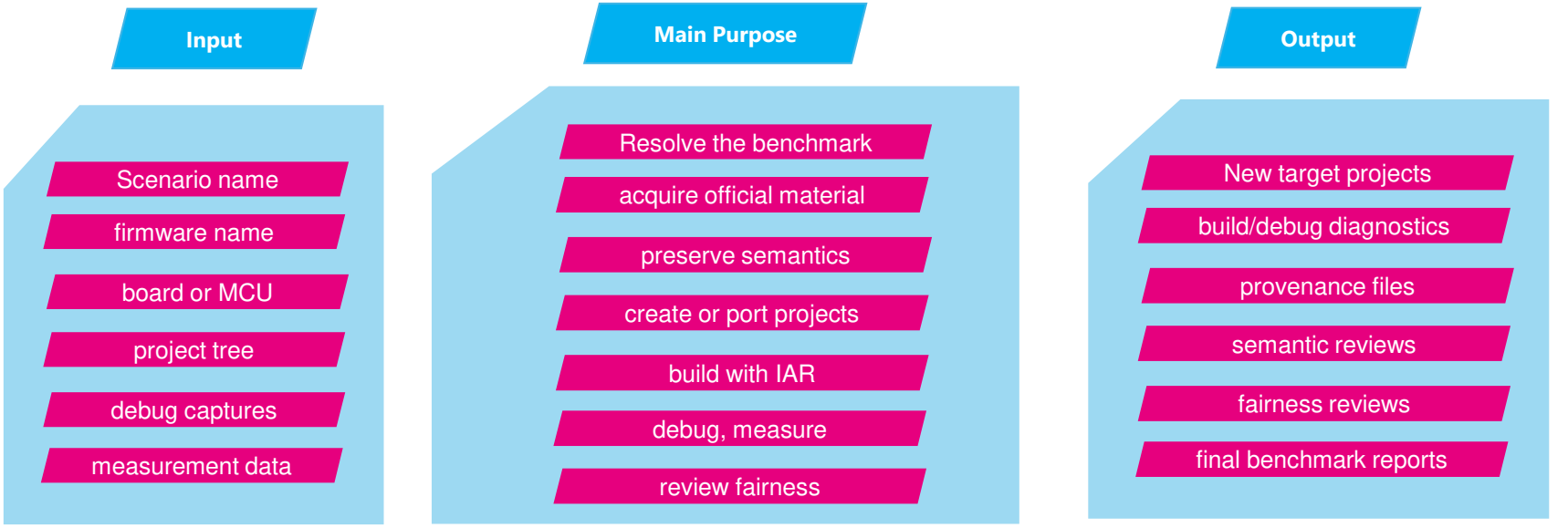
- **STM32 ROM footprint is higher by average 37% than GD32**
- **Although tests have almost the same Application and FreeRTOS stack footprint, but there is a big gap in HAL drivers' footprint (mainly due to HAL RCC footprint)**
- **FreeRTOS test have almost the same performance**

## Support Developments for the Benchmark Workflow



## Support Developments for the Benchmark Workflow

**Custom copilot agent**

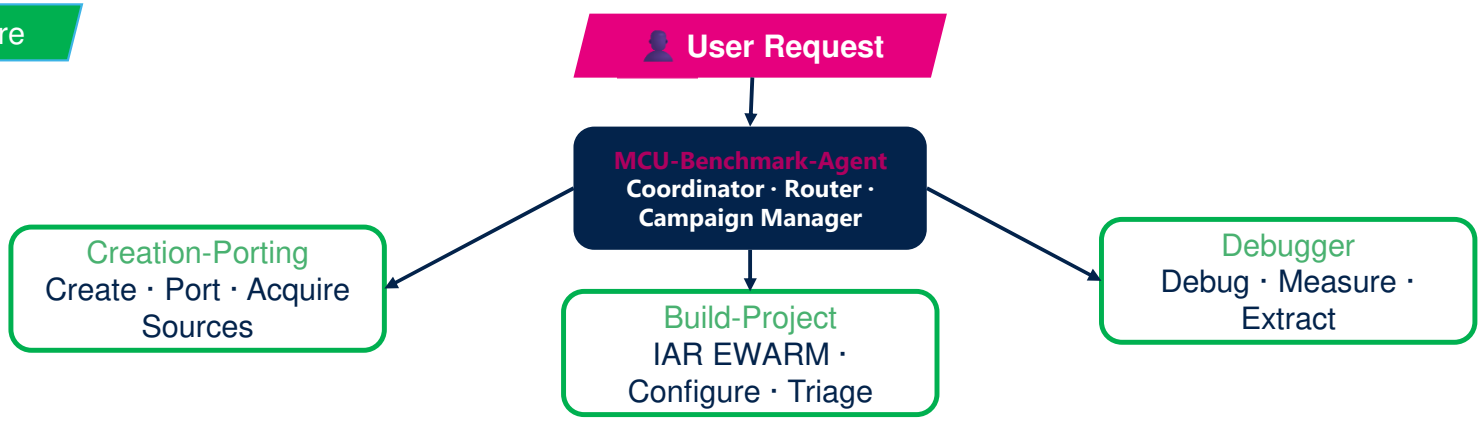



**Autonomous by default** | **Strict semantic preservation** | **Official-source provenance** | **Hardware approval gate**

## Support Developments for the Benchmark Workflow

**Custom copilot agent**

Architecture




**SKILLS**

*benchmark-analysis* | *semantic-equivalence* | *result-interpretation* | *stm32-enhancement* | *diagram-generation*

## Support Developments for the Benchmark Workflow

**Custom copilot agent**

### Use Cases

*Real scenarios you can solve with a single conversation.*

"Port this STM32 benchmark to a competitor board“
*Creates a semantically equivalent port. Tracks every adaptation (register API changes, clock init, pin mapping). Verifies that observable behavior matches the reference exactly.*

"Port competitor benchmark to STM32"
*Acquires the competitor SDK, analyzes the benchmark's behaviors, creates an equivalent STM32 project using STM32 HAL, builds it, and verifies semantic equivalence.*

"Compare these two platforms fairly"
*Extracts measurements from both, checks all fairness conditions (optimization, clock, scope, wrapper), computes per-module deltas by layer, and generates a comparison report with confidence levels.*

"Generate a benchmark report from this data"
*Produces a 13-section self-contained HTML report with executive badges, grouped bar charts, module tables, STM32-focused findings, and actionable recommendations.*

## Support Developments for the Benchmark Workflow

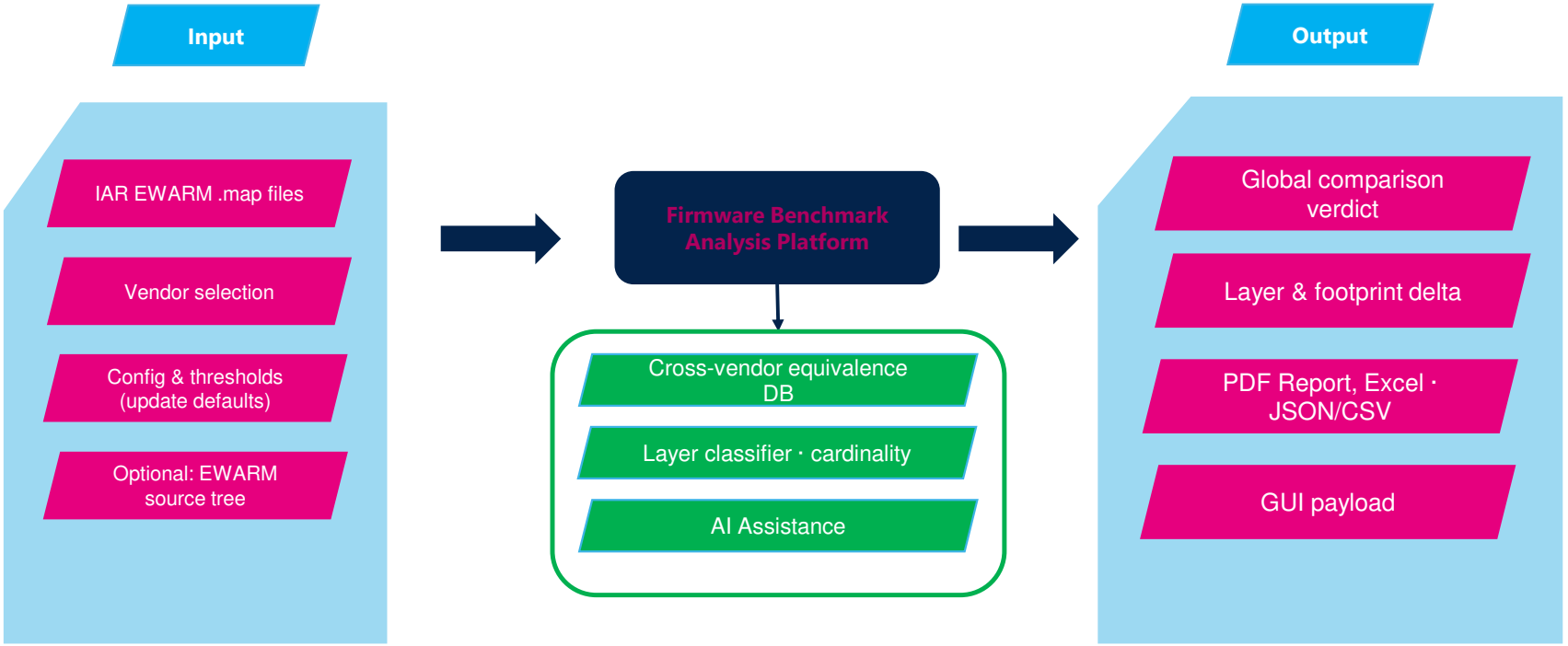
**Custom copilot agent**

### Added Value

- Turns vague benchmark requests into deterministic campaigns.
- Fetches missing firmware, CMSIS, BSP, startup, linker, and docs from official sources.
- Separates portable benchmark logic from board/vendor infrastructure.
- Builds and triages IAR EWARM projects with fix-and-retry loops.
- Checks semantic equivalence before declaring a port valid.
- Reports fairness risks before performance conclusions.

## Support Developments for the Benchmark Workflow

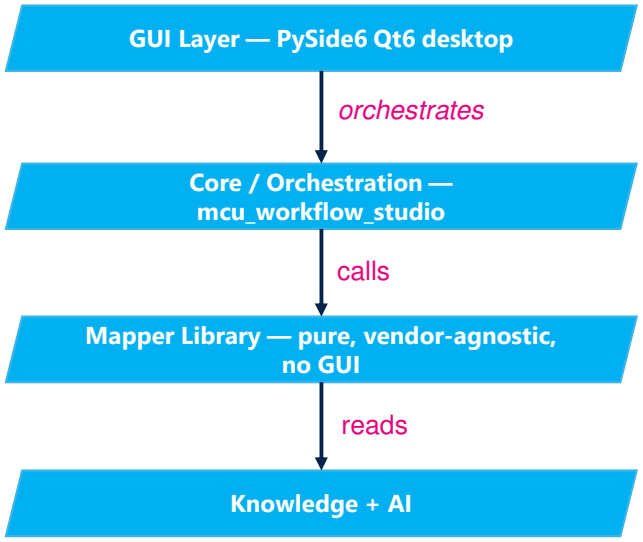
**Footprint analyzer tool**



# Support Developments for the Benchmark Workflow

**Footprint analyzer tool**

## ARCHITECTURE




## HOW IT WORKS

**1. Load Map Files**
Parse IAR EWARM .map files from source and target firmware builds

**2. Ingest Source Code (Optional)**
Parse C/H files from EWARM project dirs — extract signatures, call graphs, includes

**3. Classify Architecture**
Auto-categorize modules into Application, Middleware, HAL, Runtime layers

**4. Score Candidates**
12-signal scoring engine + source code evidence evaluates all pairings

**5. Rank, Accept & Report**
Select best matches, generate comparison, recommendations, and PDF report

# Support Developments for the Benchmark Workflow

**Footprint analyzer tool**

*Added Value, Advantages, and Limitations*

**Main Value**

**Automation**
Transforms raw map files into interpretable benchmark information.

**Main Advantage**

**Reproducibility**
Provides a more consistent and explainable comparison workflow.

**Main Limitation**

**Scope Bound**
Focused on static footprint analysis and current input/toolchain constraints.

**Added Value**

**Automation**
What this tool brings to the benchmark workflow.

- Reduces the manual effort of reading and comparing linker map files
- Provides a **layer-aware** and **module-aware** footprint breakdown
- Makes the footprint gap easier to interpret and communicate

**Advantages**

**Reproducibility**
Why it is useful from an engineering point of view.

- **Deterministic core**: same inputs lead to the same analysis outcome
- **Explainable mapping**: symbol and module correspondences are traceable
- **Cross-vendor orientation**: designed specifically for competitor comparison

**Limitations**

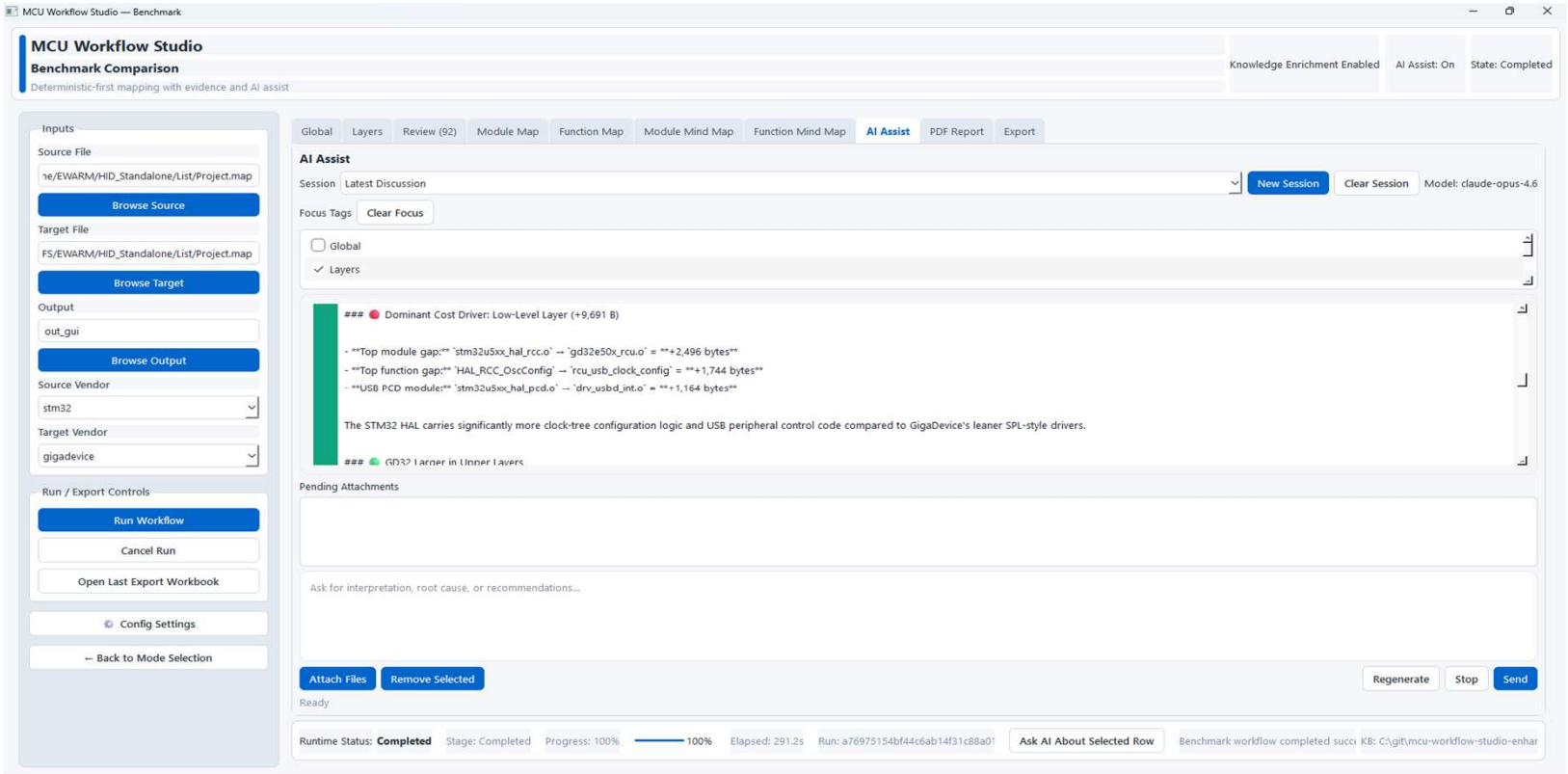
**Scope Bound**
Current boundaries of the tool.

- Focused on **IAR EWARM linker maps**; other toolchains are not yet covered
- Performs **static footprint analysis**, not runtime stack/heap behavior measurement
- Quality scores and improvement hints are still partly heuristic

## Support Developments for the Benchmark Workflow

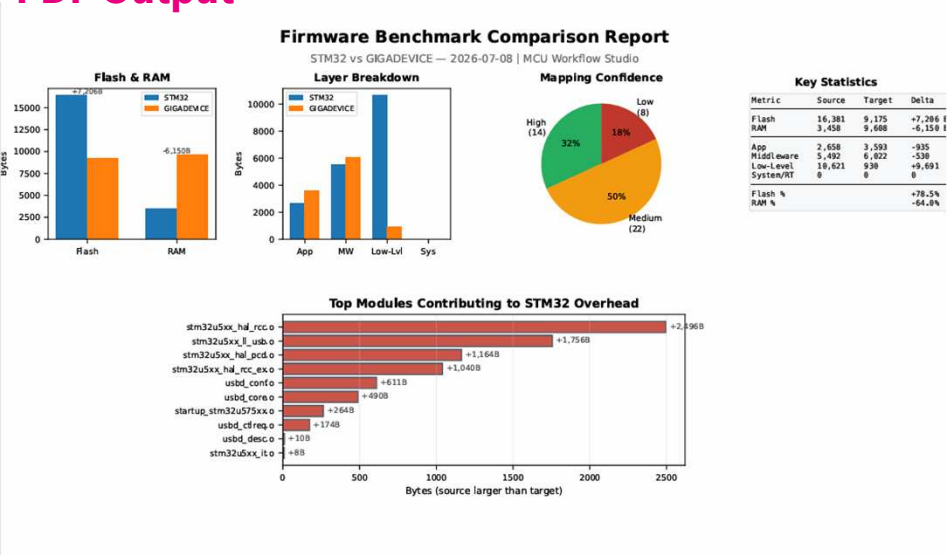
**Footprint analyzer tool**

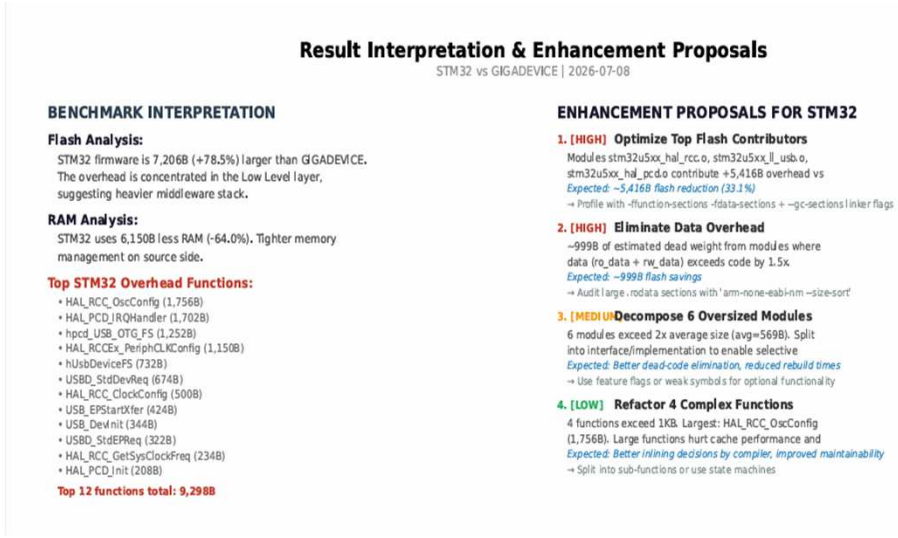
**Demo**



# Support Developments for the Benchmark Workflow

## PDF Output



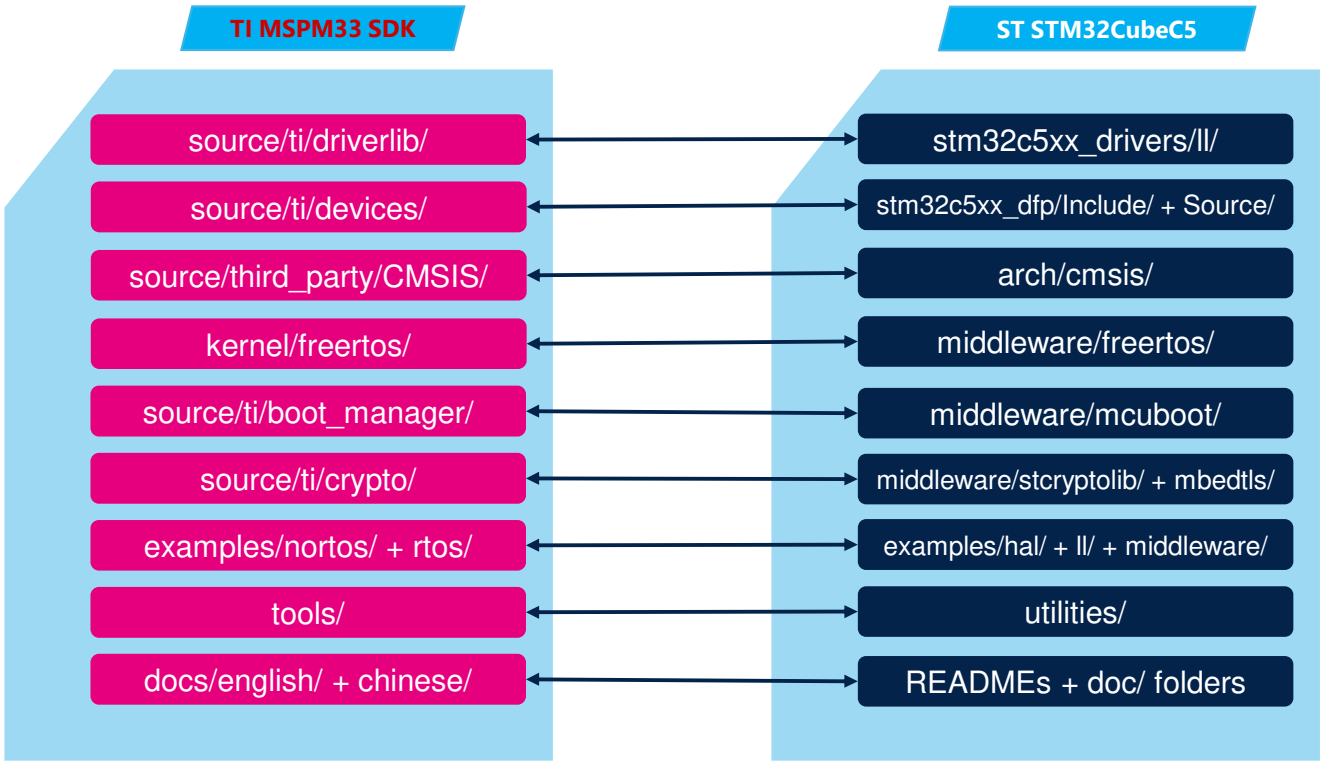



Static analysis

# STM32Cube 2 (C5) vs TI MSPM33

Software suit TI vs ST: Firmware Library analysis

## SDK Architecture Mapping



Static analysis

# STM32Cube 2 (C5) vs TI MSPM33

## Architectural Context

**TI MSPM33 SDK**

**Driverlib**
- Single driverlib layer — direct register manipulation
- No abstraction hierarchy or portability layer
- Minimal startup & configuration overhead
- Locked to MSPM33 family — no cross-platform reuse
- Fewer safety assertions — leaner but less defensive
- Demo-oriented examples — less production boilerplate

**ST STM32CubeC5**

**Layered Abstraction**
- **HAL:** Full abstraction with callbacks, error handling, state machines
- **LL:** Thin inline wrappers over registers — minimal overhead
- CMSIS-compliant startup & SystemClock_Config included
- Portable across 1,000+ STM32 variants without source changes
- MISRA C compliant — defensive coding adds safety guarantees
- Production-ready error handlers & assert framework

# STM32Cube 2 (C5) vs TI MSPM33

## Middleware & Software Stack Gaps

**Static analysis**

### TI MSPM33 — Exclusive Middleware

| | |
|---|---|
| Edge AI (ML Inference) | AI/ML |
| LVGL (Graphics/GUI) | Display |
| LIN Bus Protocol | Auto |
| DALI (Smart Lighting) | Lighting |
| IQmath (Fixed-Point DSP) | Math |
| GUI Composer | Viz |
| Smoke Detection Algorithm | Safety |
| Post-Quantum Crypto (ML-DSA) | PQC |
| Character Recognition | OCR |

### Common Middleware

- **FreeRTOS**
  - kernel/freertos/
  - middleware/freertos/
- **MCUboot (Secure Boot)**
  - source/third_party/mcuboot/
  - middleware/mcuboot/
- **Edge AI (ML Inference)**
  - source/ti/crypto/ + boot_manager/sha2sw/
  - middleware/stcryptolib/ + mbedtls/

### ST STM32C5 — Exclusive Middleware

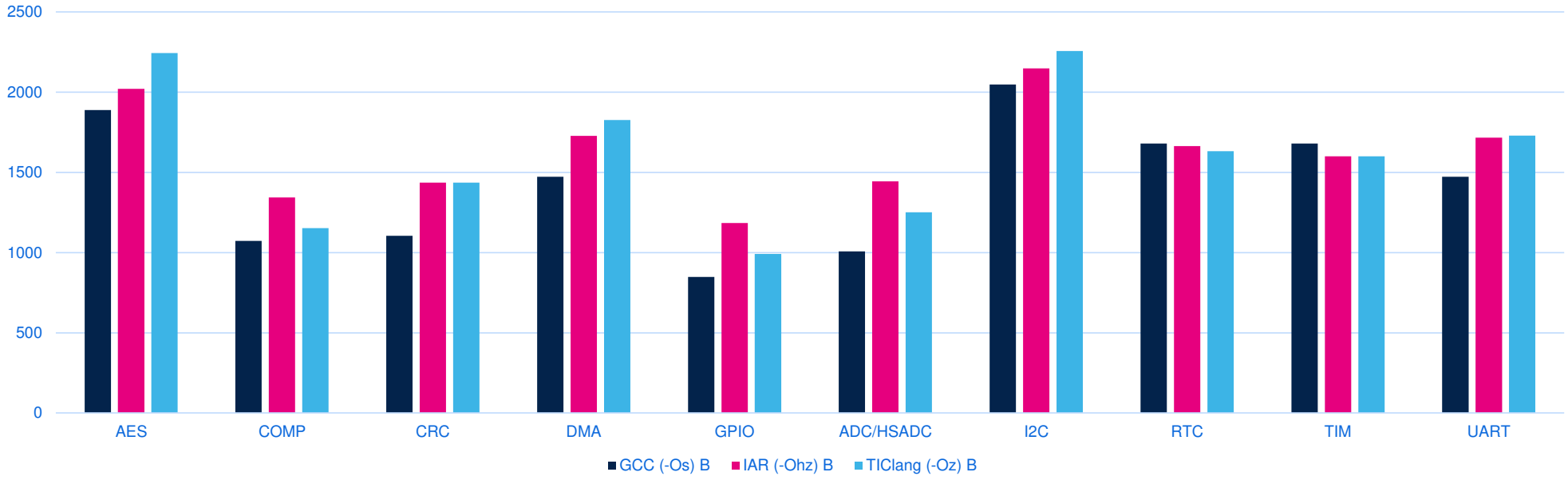
| | |
|---|---|
| FileX (File System) | FAT |
| LevelX (Wear Leveling) | NOR/NAND |
| LwIP (TCP/IP Stack) | IPv4/v6 |
| USBX (USB Classes) | CDC/HID/MSC |
| mbedTLS (TLS/SSL) | TLS 1.3 |
| Open Bootloader | Multi-IF |
| stFCF (FW Config Framework) | Prov. |
| stCryptoLib (Extended) | 30+ algos |

## STM32Cube 2 (C5) vs TI MSPM33

**Compiler Optimization difference**

**Static analysis**

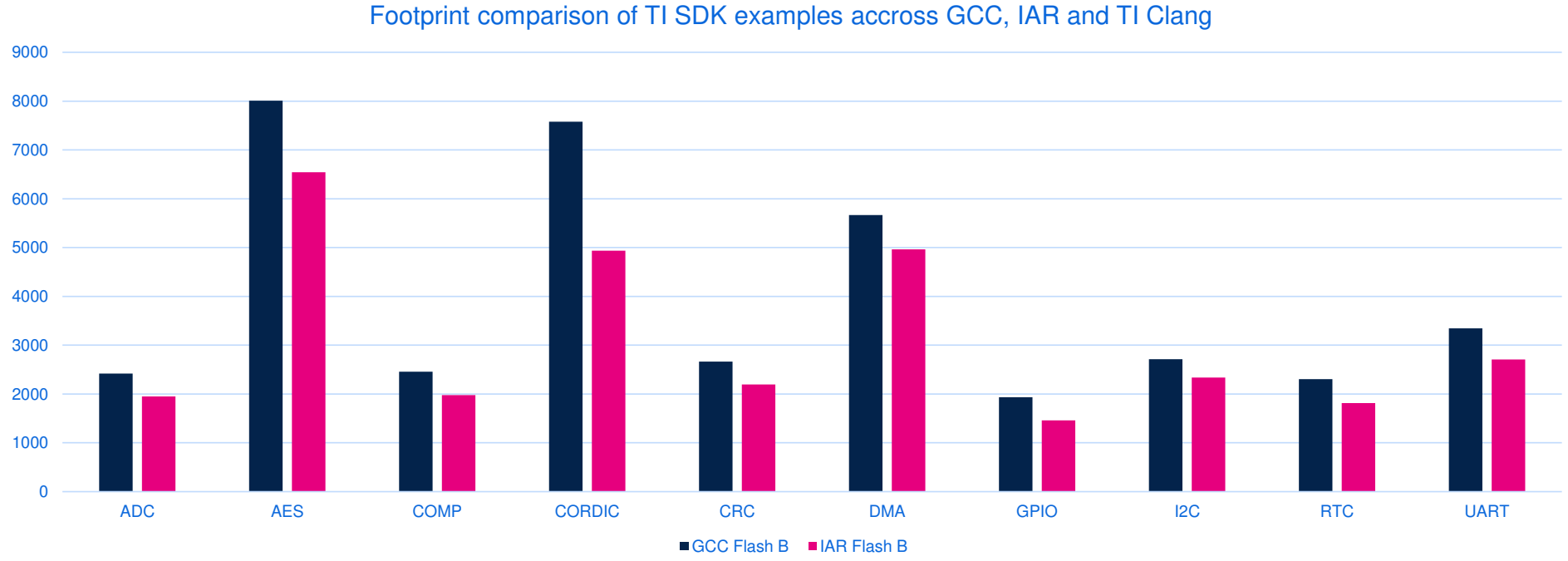
Footprint comparison of STM32C5 examples accross GCC, IAR and TI Clang



## STM32Cube 2 (C5) vs TI MSPM33

**Compiler Optimization difference**

**Static analysis**



# STM32Cube 2 (C5) vs TI MSPM33

## Low level project comparison

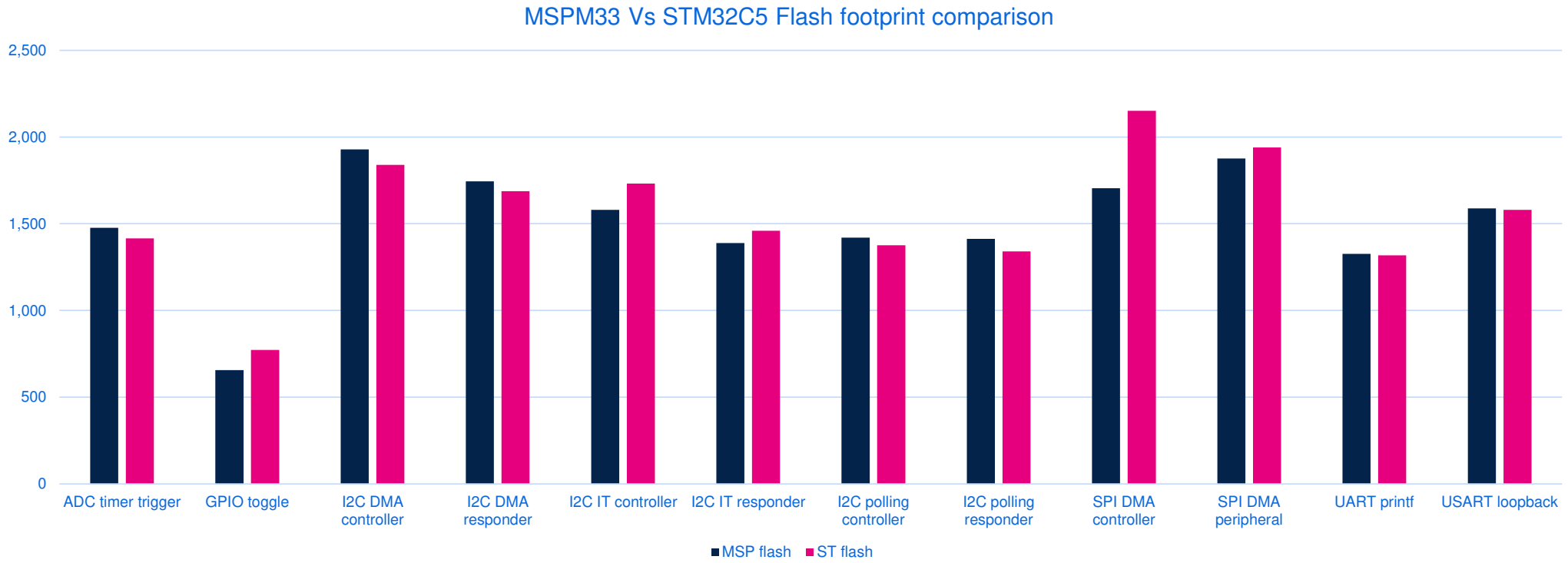
Functionnal analysis

| Scenario | MSP RO code | MSP RO data | MSP flash | ST RO code | ST RO data | ST flash | ST flash delta bytes | ST flash delta % | MSP RAM | ST RAM | ST RAM delta bytes | ST RAM delta % |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **ADC timer trigger** | 1,120 | 356 | 1,476 | 940 | 476 | 1,416 | -60 | -4.10% | 513 | 513 | 0 | 0.00% |
| **GPIO toggle** | 380 | 276 | 656 | 312 | 460 | 772 | 116 | 17.70% | 512 | 512 | 0 | 0.00% |
| **I2C DMA controller** | 1,512 | 416 | 1,928 | 1,312 | 528 | 1,840 | -88 | -4.60% | 553 | 517 | -36 | -6.50% |
| **I2C DMA responder** | 1,288 | 456 | 1,744 | 1,164 | 523 | 1,687 | -57 | -3.30% | 593 | 556 | -37 | -6.20% |
| **I2C IT controller** | 1,208 | 372 | 1,580 | 1,216 | 516 | 1,732 | 152 | 9.60% | 557 | 514 | -43 | -7.70% |
| **I2C IT responder** | 976 | 412 | 1,388 | 944 | 516 | 1,460 | 72 | 5.20% | 600 | 553 | -47 | -7.80% |
| **I2C polling controller** | 1,052 | 368 | 1,420 | 860 | 516 | 1,376 | -44 | -3.10% | 556 | 513 | -43 | -7.70% |
| **I2C polling responder** | 992 | 420 | 1,412 | 824 | 516 | 1,340 | -72 | -5.10% | 608 | 552 | -56 | -9.20% |
| **SPI DMA controller** | 1,240 | 464 | 1,704 | 1,632 | 519 | 2,151 | 447 | 26.20% | 592 | 556 | -36 | -6.10% |
| **SPI DMA peripheral** | 1,412 | 464 | 1,876 | 1,428 | 512 | 1,940 | 64 | 3.40% | 592 | 552 | -40 | -6.80% |
| **UART printf** | 1,010 | 316 | 1,326 | 858 | 460 | 1,318 | -8 | -0.60% | 512 | 512 | 0 | 0.00% |
| **USART loopback** | 1,192 | 396 | 1,588 | 1,080 | 500 | 1,580 | -8 | -0.50% | 532 | 532 | 0 | 0.00% |

# STM32Cube 2 (C5) vs TI MSPM33

**Low level project comparison**

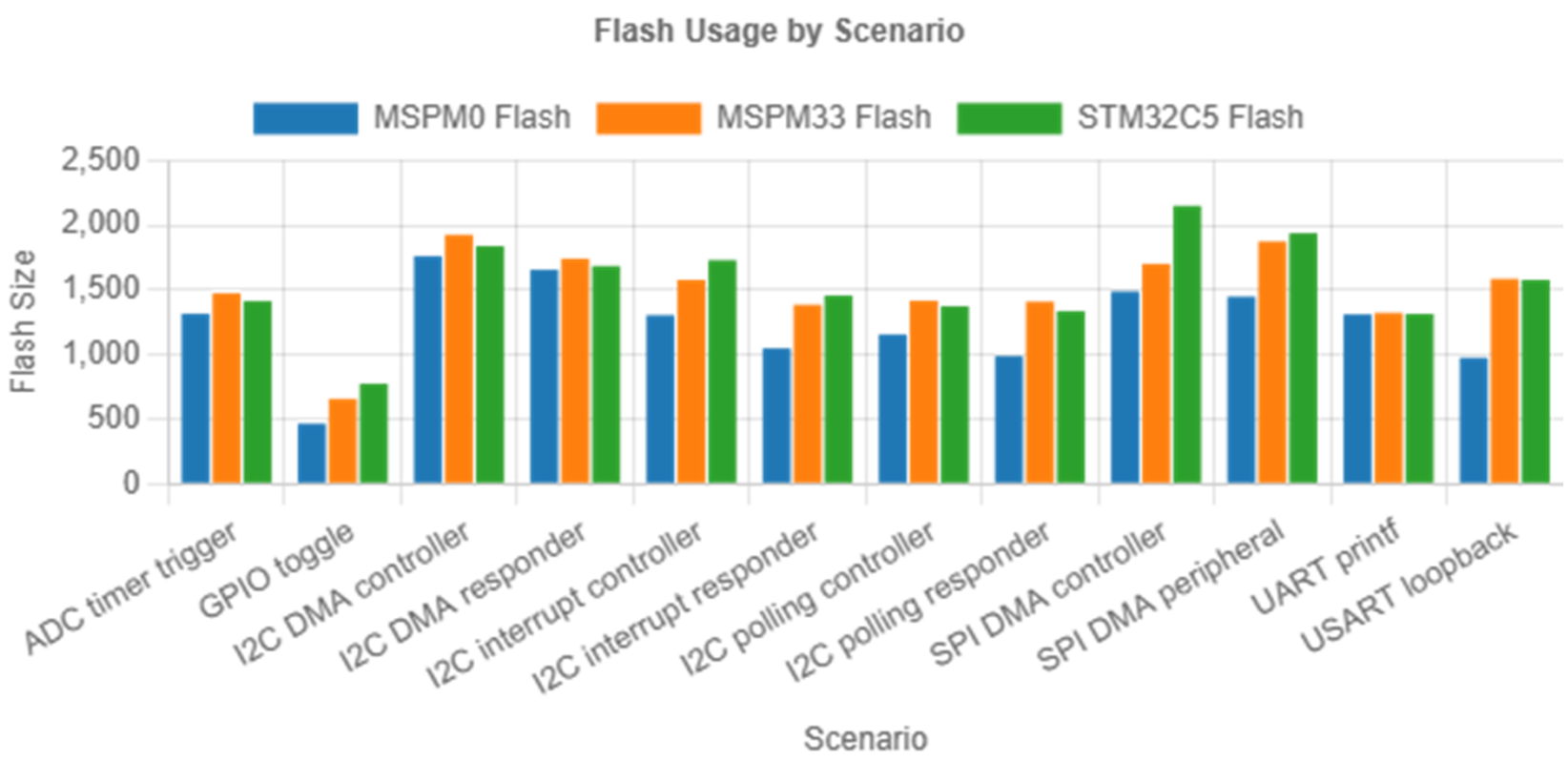
**Functional analysis**



## STM32Cube 2 (C5) vs TI MSPM33

**Low level project comparison**

**Functional analysis**



STM32Cube 2 (C5) vs TI MSPM33

Low level project comparison

Functional analysis

## Aggregate Results

| Metric | MSPM33 total | STM32C5 total | Delta (bytes) | Delta (%) | Interpretation |
|---|---|---|---|---|---|
| Flash | 18,098 | 18,612 | +514 | +2.8% | MSPM33 has the smaller aggregate linked image |
| RAM | 6,720 | 6,382 | -338 | -5.0% | STM32 has the smaller aggregate RAM allocation |

**Scorecard**

- **Flash wins:** STM32 7, MSPM33 5
- **RAM wins:** STM32 8, MSPM33 0
- **RAM ties:** 4

**Interpretation**

- STM32 is stronger on RAM efficiency.
- MSPM33 remains slightly better on aggregate flash.
- STM32 is competitive in most scenario-level flash comparisons.

# STM32Cube 2 (C5) vs TI MSPM33

**Low level project comparison**

**Functional analysis**

## Where STM32 Is Strong

**Scenario wins**

- ADC timer trigger
- I2C DMA controller
- I2C DMA responder
- I2C polling controller
- I2C polling responder
- UART printf
- USART loopback

**Main reasons**

- Minimal app-owned initialization
- Removal of unused buffers and dead state
- Lower RW data in many communication scenarios

**Key point**

STM32 is not inherently worse in executable code size. In some scenarios, STM32 RO code is smaller than MSPM33 RO code. Several remaining flash losses come from fixed RO-data overhead rather than inefficient application logic.

STM32Cube 2 (C5) vs TI MSPM33

**Low level project comparison**

**Functional analysis**

## Where STM32 Is Weaker

**GPIO toggle**

- Flash delta: **+116 bytes**
- Flash delta: **+17.7%**
- RAM: tie

**I2C interrupt roles**

- IT controller: **+152 bytes**, **+9.6%**
- IT responder: **+72 bytes**, **+5.2%**
- But STM32 still wins RAM in both

**SPI DMA**

- Controller: **+447 bytes**, **+26.2%**
- Peripheral: **+64 bytes**, **+3.4%**
- STM32 still wins RAM in both

**Main causes**

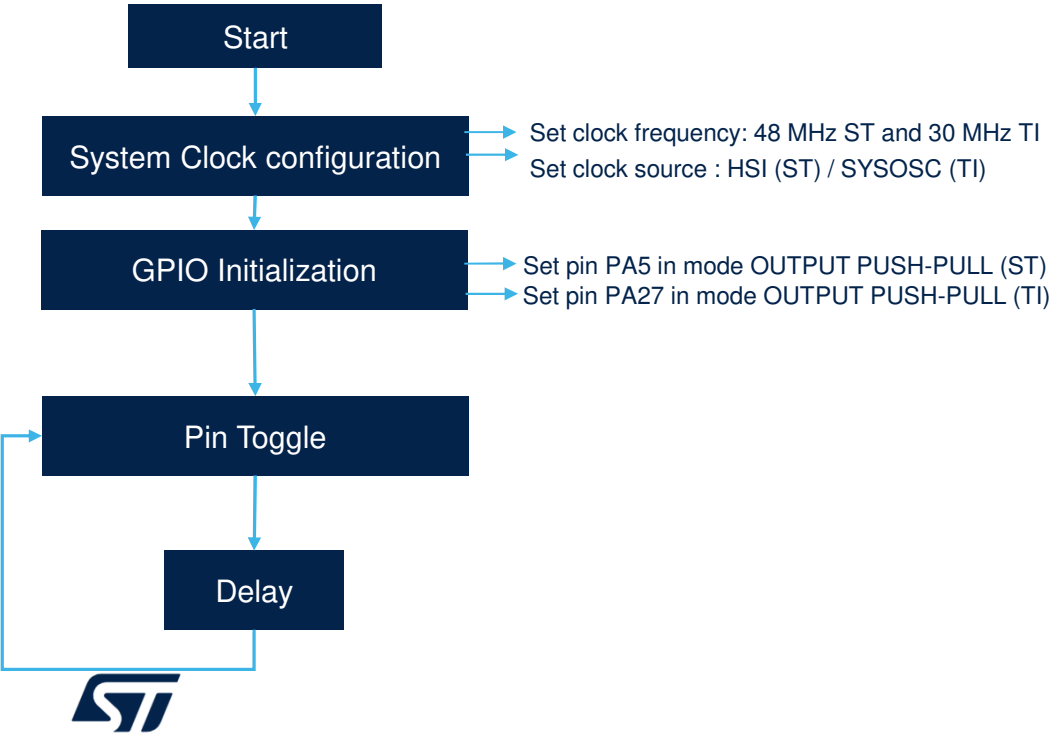
- Larger fixed startup/vector-table RO-data footprint
- Required interrupt event/error infrastructure

# STM32Cube 2 (C5) vs TI MSPM33

**Low level project comparison**

**Functional analysis**

## Case Study – GPIO-Toggle




**Observed result**

- MSPM33 flash: **656**
- STM32 flash: **772**
- Delta: **+116 bytes**, **+17.7%**

**Important nuance**

- MSPM33 RO code: **380**
- STM32 RO code: **312**
- STM32 code is actually **68 bytes smaller**

**Why STM32 still loses**

- MSPM33 RO data: **276**
- STM32 RO data: **460**
- Fixed RO-data difference: **184 bytes**

This is a structural startup/vector-table penalty. The GPIO case is too small for the lower STM32 code size to overcome the fixed RO-data overhead.

STM32Cube 2 (C5) vs TI MSPM33

Low level project comparison

Functional analysis

## Case Study — I2C Family

| Scenario | Flash delta (bytes) | Flash delta (%) | RAM delta (bytes) | RAM delta (%) | Interpretation |
|---|---|---|---|---|---|
| I2C DMA controller | -88 | -4.6% | -36 | -6.5% | STM32 flash/RAM win |
| I2C DMA responder | -57 | -3.3% | -37 | -6.2% | STM32 flash/RAM win |
| I2C IT controller | +152 | +9.6% | -43 | -7.7% | MSP flash; STM32 RAM |
| I2C IT responder | +72 | +5.2% | -47 | -7.8% | MSP flash; STM32 RAM |
| I2C polling controller | -44 | -3.1% | -43 | -7.7% | STM32 flash/RAM win |
| I2C polling responder | -72 | -5.1% | -56 | -9.2% | STM32 flash/RAM win |

**Interpretation**

- DMA and polling modes benefit strongly from cleanup of generic init and dead state.
- Interrupt mode still pays flash for event/error setup and handlers.
- Result: STM32 is strong on RAM throughout I2C, but only mixed on flash in interrupt mode.

# STM32Cube 2 (C5) vs TI MSPM33

**Functional analysis**

## Case Study — SPI DMA

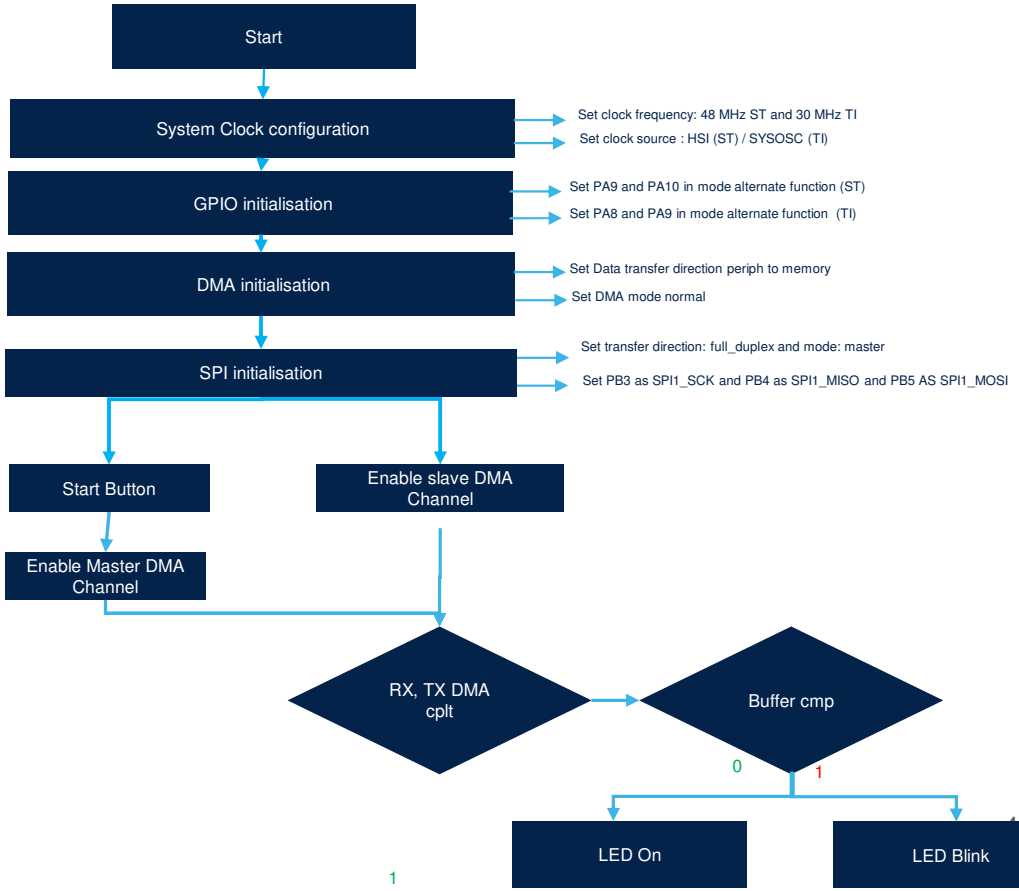
**Low level project comparison**

**Controller**

- MSPM33 flash: **1,704**
- STM32 flash: **2,151**
- Delta: **+447 bytes**, **+26.2%**
- RAM delta: **-36 bytes**, **-6.1%**

**Peripheral**

- MSPM33 flash: **1,876**
- STM32 flash: **1,940**
- Delta: **+64 bytes**, **+3.4%**
- RAM delta: **-40 bytes**, **-6.8%**




**Why STM32 is worse here**

The STM32 application explicitly carries GPIO, DMA, SPI, NVIC, EOT, DMA-error, compare, LED, and delay behavior in the application object. MSPM33 resolves more of this behavior through compact linked DriverLib modules. This remains STM32's largest flash weakness in the portfolio.

# STM32Cube 2 (C5) vs TI MSPM33

**Low level project comparison**

**Functional analysis**

## MSPM0 Extension

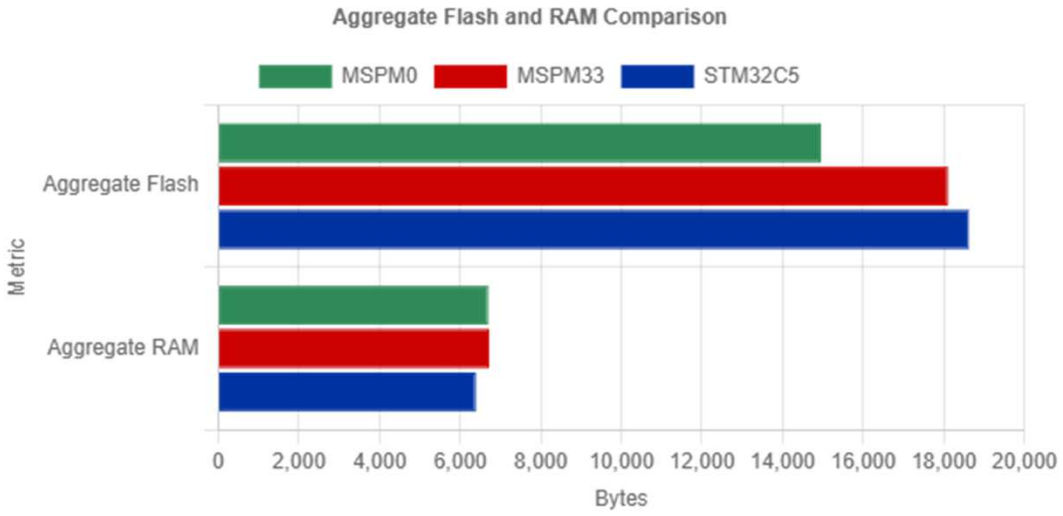
| Metric | MSPM0 total | MSPM33 total | STM32C5 total |
|---|---|---|---|
| Flash | 14,954 | 18,098 | 18,612 |
| RAM | 6,692 | 6,720 | 6,382 |

**Derived deltas vs MSPM0**

- STM32 flash: **+3,658 bytes**, **+24.5%**
- STM32 RAM: **-310 bytes**, **-4.6%**
- MSPM33 flash: **+3,144 bytes**, **+21.0%**
- MSPM33 RAM: **+28 bytes**, **+0.4%**

**Interpretation**

- MSPM0 has the smallest aggregate flash of the three platforms.
- STM32 keeps the smallest aggregate RAM.
- The portfolio therefore splits into a flash leader and a RAM leader.

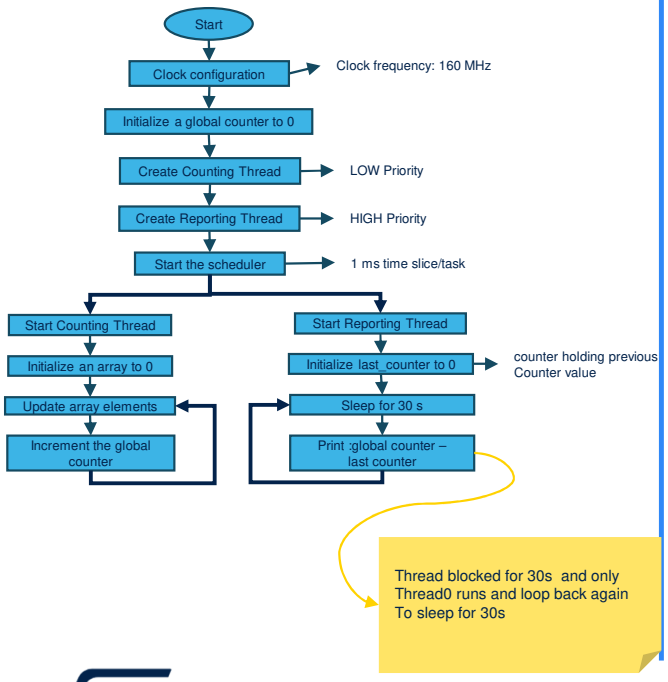


# STM32Cube 2 (C5) vs TI MSPM33

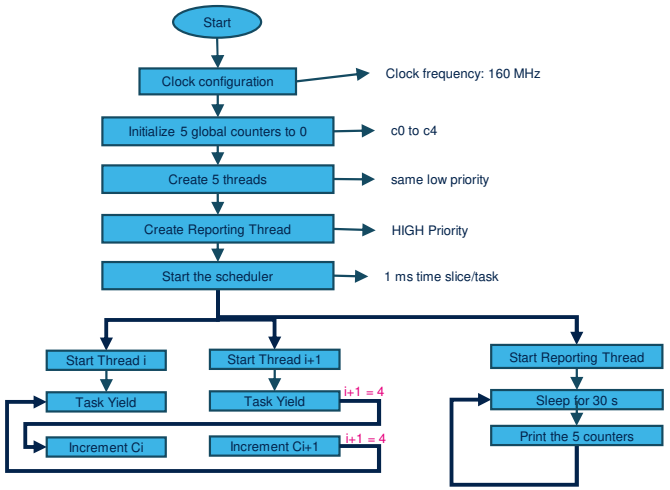
## Middleware benchmark : FreeRTOS

**Functional analysis**

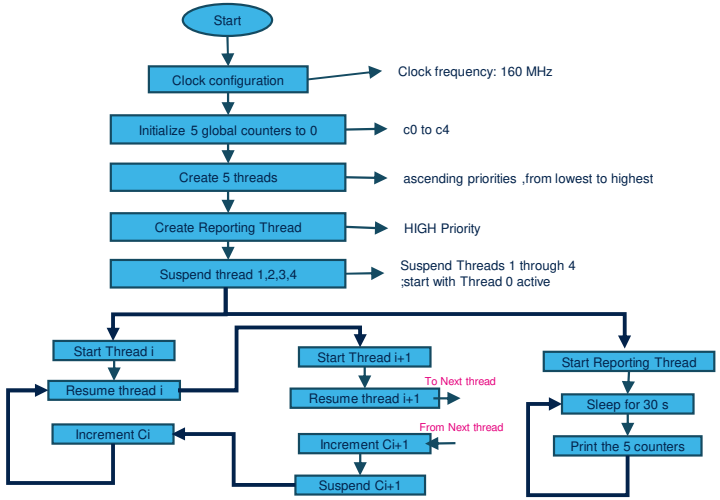
### Basic Processing Flowchart




### Cooperative Processing Flowchart




### Preemptive Scheduling Flowchart



# STM32Cube 2 (C5) vs TI MSPM33

**Middleware benchmark : FreeRTOS**

**Functional analysis**

## Executive Summary

Across the 3 FreeRTOS benchmark scenarios, STM32C5 consistently produces the smaller linked flash image, with a measured reduction of about 28–30% versus TI MSPM33 after TI's FreeRTOS configuration was aligned to remove an avoidable fairness artifact. RAM is effectively tied, with STM32 only 56–76 bytes higher depending on scenario.

Observed runtime logs also show higher raw throughput on STM32 in all 3 scenarios. However, that runtime result must be treated as observed behavior only, not a final capability conclusion, because CPU clock parity and cross-target codegen equivalence were not fully verified.

| FLASH OUTCOME | AVERAGE FLASH ADVANTAGE | RAM OUTCOME | RUNTIME CONCLUSION STATUS |
|---|---|---|---|
| STM32 Smaller | -28.6% | Near Tie | Not Final |
| All 3 scenarios, post-alignment | Range: -27.9% to -29.6% | STM32 +56 to +76 bytes | Observed throughput only |

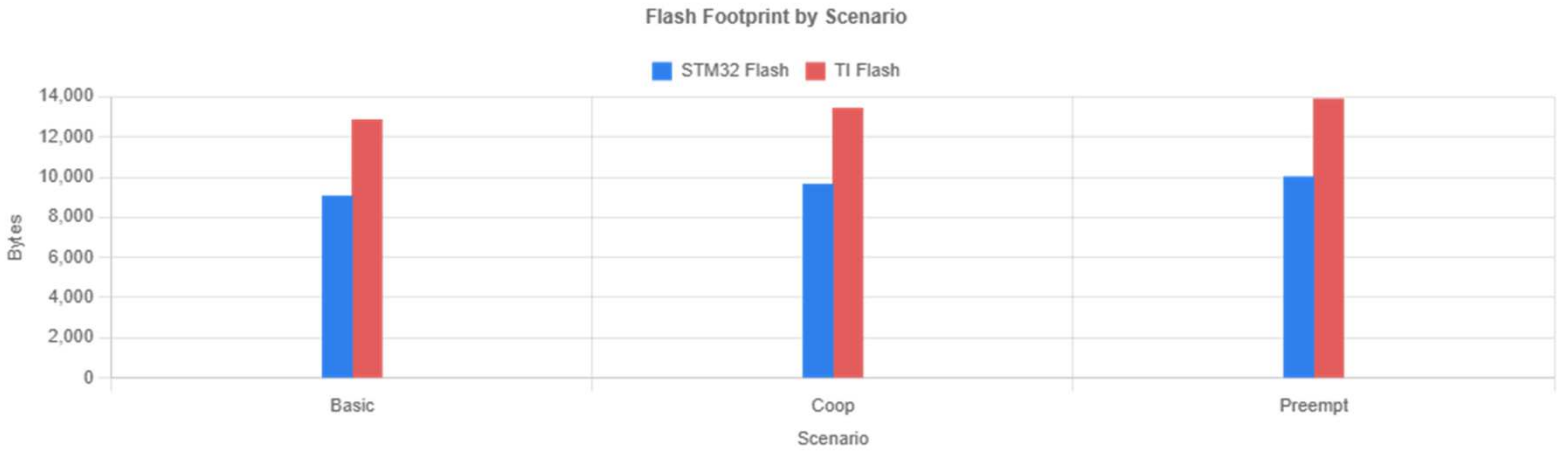
**Bottom line: For this FreeRTOS benchmark campaign, the strongest confirmed result is flash footprint leadership for STM32C5. The application layer is close to equivalent across vendors, while the remaining footprint gap is driven primarily by TI's low-level/library side, especially vsnprintf()-driven full printf inclusion.**

# STM32Cube 2 (C5) vs TI MSPM33

**Middleware benchmark : FreeRTOS**

**Functional analysis**

| SCENARIO | STM32 FLASH | TI FLASH | FLASH DELTA | STM32 RAM | TI RAM | RAM DELTA | RESULT |
|---|---|---|---|---|---|---|---|
| basic_processing | 9,052 | 12,854 | -3,802 (-29.58%) | 10,196 | 10,140 | +56 (+0.55%) | STM32 flash win |
| cooperative_scheduling | 9,640 | 13,430 | -3,790 (-28.22%) | 9,188 | 9,132 | +56 (+0.61%) | STM32 flash win |
| preemptive_scheduling | 10,012 | 13,894 | -3,882 (-27.94%) | 9,188 | 9,112 | +76 (+0.83%) | STM32 flash win |

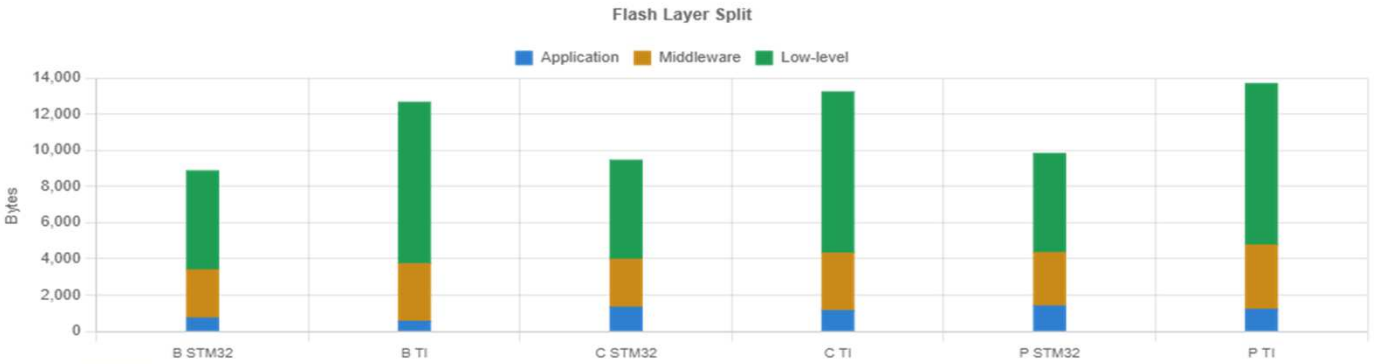


# STM32Cube 2 (C5) vs TI MSPM33

## Middleware benchmark : FreeRTOS

**Functional analysis**

| Scenario | Layer | Flash STM32 | Flash TI | Flash Delta | RAM STM32 | RAM TI | RAM Delta |
|---|---|---|---|---|---|---|---|
| Basic processing | Application | 736 | 558 | +178 | 1,028 | 1,028 | 0 |
| Basic processing | Middleware | 2,669 | 3,186 | -517 | 8,572 | 8,596 | -24 |
| Basic processing | Low-level | 5,479 | 8,934 | -3,455 | 596 | 516 | +80 |
| Cooperative scheduling | Application | 1,332 | 1,150 | +182 | 20 | 20 | 0 |
| Cooperative scheduling | Middleware | 2,661 | 3,178 | -517 | 8,572 | 8,596 | -24 |
| Cooperative scheduling | Low-level | 5,479 | 8,926 | -3,447 | 596 | 516 | +80 |
| Preemptive scheduling | Application | 1,400 | 1,218 | +182 | 20 | 0 | +20 |
| Preemptive scheduling | Middleware | 2,963 | 3,566 | -603 | 8,572 | 8,596 | -24 |
| Preemptive scheduling | Low-level | 5,481 | 8,934 | -3,453 | 596 | 516 | +80 |



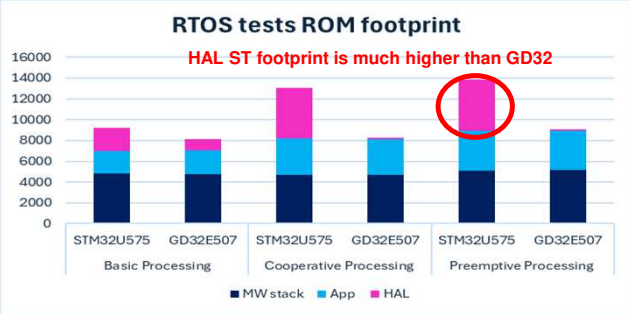

The benchmark application logic is essentially comparable across vendors. In multiple cases, object-level evidence shows only a few bytes of difference, supporting that the app itself is **not the main driver** of the measured gap.
A significant fairness artifact on TI was removed by aligning FreeRTOS feature flags with STM32. That reduced the middleware gap from roughly **46–50%** down to roughly **16–17%**.
The largest remaining flash delta is dominated by TI's `vsnprintf()` path, which pulls in a much larger full printf formatter. This is the main remaining explanation for the measured flash difference.

# Results and Contributions

## FreeRTOS tests Footprint

Porting tests from STM32 (existing) to GD32 is performed by Copilot + Manual validation and execution on board

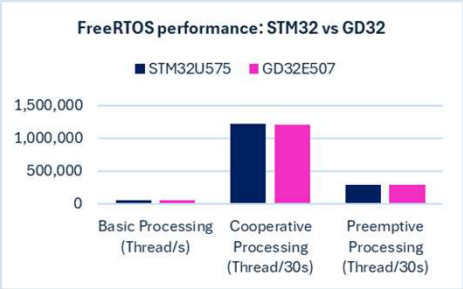
| RTOS Scenario | Footprint | STM232 vs GD32 |
|---|---|---|
| Basic Processing | ROM | + 40.86 % |
| | RAM | +2.08 % |
| Cooperative Processing | ROM | + 50 % |
| | RAM | +2.31 % |
| Preemptive Processing | ROM | + 46 % |
| | RAM | + 2.31 % |




- Although STM32 and GD32 have same RAM consumption, but there is a significant gap in ROM footprint.
- ROM footprint splitting shows that STM32 and GD32 tests have same Application footprint (same scenario) and FreeRTOS footprint (same stack), however the STM32 HAL drivers' footprint is much higher than GD32, mainly due to STM32 HAL RCC driver footprint

## FreeRTOS tests performance (thread/s)

| RTOS Scenario | STM232 vs GD32 |
|---|---|
| Basic Processing (Thread/s) | - 0.025 % |
| Cooperative Processing (Thread/30s) | + 1.6 % |
| Preemptive Processing (Thread/30s) | + 3.3 % |

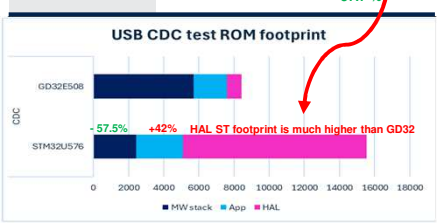



STM32 and GD32 FreeRTOS tests have almost the same performance

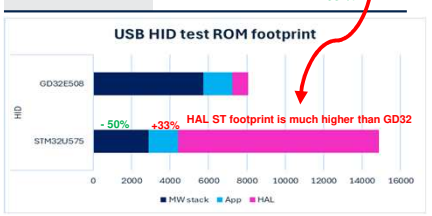
## STM32/GD32 USB Library Footprint

Porting tests from STM32 (existing) to GD32 is performed by Copilot + Manual validation and execution on board

| | Footprint | STM232 vs GD32 |
|---|---|---|
| CDC Device (FS) | ROM | + 88 % |
| | RAM | - 57.7 % |




| | Footprint | STM232 vs GD32 |
|---|---|---|
| HID Device (FS) | ROM | + 77 % |
| | RAM | - 65 % |




- The ST STM32 USB device MW stack ROM footprint is lower than GD32 USB device MW stack by 57.5% in VDV Device and 50% in HID Device because USB Device configuration is performed at the application level, whereas for GD32 it is managed in the middleware stack.
- STM32 total ROM footprint is higher by 88% in CDC Device and 77% in HID Device than GD32. This is due to:
  - **Application level**: STM32 higher than GD32, the STM32 USB configuration represents 43% in CDC Device and 38% in HID Device of the total application (user files: usb_device.c, usbd_cdc_if.c, usbd_desc.c, usbd_conf.c)
  - **HAL level**: STM32 is higher than GD32, mainly due to:
    - HAL RCC driver: 4560 bytes (44% of total HAL)
    - HAL/LL USB driver: 5006 bytes (48% of total HAL)

## STM32/GD32 USB Library Execution time (ms)

| USB CDC operation | STM232 vs GD32 |
|---|---|
| Initialization | -61.68% |
| Enumeration | -0.14% |
| Transmit data | -1.53% |
| Receive data | -0.72% |

| USB HID operation | STM232 vs GD32 |
|---|---|
| Initialization | -61.70% |
| Enumeration | -1.39% |
| Data transmission | -0.71% |

STM32 USB Device **outperforms GD32 by 61%** in initialization phase.
STM32 and GD32 USB Device tests have almost the same execution time in Enumeration, data transmission and reception phases.

## Takeaway

- **FreeRTOS footprint is comparable** between STM32 and GD32 because both platforms use **the same RTOS stack**, resulting in a similar middleware layer footprint.
- In the **USB middleware**, ST's native USB library shows **a clear footprint advantage**, with approximately **half the ROM footprint** compared to the native GD32 USB library.
- **The HAL footprint results confirm that the main contributor of the ROM gap is the STM32 RCC driver,** which is significantly larger and increases the overall footprint of ST examples versus competitors.
- These results are consistent with the message **already shared with Marketing** and further justify **the move toward Cube 2.0**, where ST has optimized its drivers to reduce footprint.

# Perspectives and conclusion

## Conclusion

•This PFE focused on **benchmarking the STM32Cube ecosystem against competitor ecosystems** in an industrial context.
•A **structured and reproducible benchmark methodology** was defined to ensure fair cross-vendor comparison.
•Legacy benchmark scenarios were **rebuilt, updated, and exploited again**, especially for USB comparisons.
•The work was extended to **new benchmark directions**, notably:
- **ST vs GD32 middleware benchmark**
- **STM32Cube 2 (C5) vs TI MSPM33 static analysis**

•The benchmark results made it possible to:
- **highlight STM32Cube strengths**
- **identify technical gaps**
- **propose concrete enhancement directions**

•In parallel, support developments were created to **improve benchmark execution, analysis, and reproducibility**.

## Perspectives

•Extend the **ST vs GD32 benchmark** to additional middleware scenarios and deeper analysis.
•Continue the **STM32Cube 2 (C5) vs TI MSPM33** study with:
- runtime benchmark scenarios
- functional comparison
- footprint and execution-time measurements

•Enrich the benchmark coverage toward:
- more software stacks
- more vendors
- more use cases

•Improve the benchmark workflow by:
- extending tool support to more configurations and toolchains
- strengthening result automation and reporting
- reusing the developed workflow in future benchmark campaigns

•Use benchmark outcomes to guide **future STM32Cube enhancement actions**.

# Our technology starts with You

Find out more at www.st.com

For further support in creating a PowerPoint presentation, including graphic assets, formatting tools and additional information on the ST brand

**you can visit the ST Brand Portal**

https://brandportal.st.com